EN.553.636 Introduction to Data Science

Jonathan Y. Ma

June 2026

This course was taught by Dr. Tamas Budavari in the Fall 2025 Semester. The notes are transcribed by Jonathan Ma and may contain errors and omitted content, so any issues are to be directed towards me.

Contents

1 Descriptive Statistics **8**
- 1.1 Data Sets . . . 8
- 1.2 Measures of Location . . . 8
- 1.3 Measures of Dispersion . . . 9
- 1.4 Outliers . . . 9
- 1.5 Robustness . . . 10

2 Distributions **11**
- 2.1 Probability Density Functions . . . 11
- 2.2 Cumulative Distribution Function . . . 11
- 2.3 Expectation and Moments . . . 12
- 2.4 Python Introduction . . . 13
  - 2.4.1 Core Language Concepts . . . 13
  - 2.4.2 Pandas Introduction . . . 14
  - 2.4.3 Seaborn Introduction . . . 14

3 Samples and Kernels **15**
- 3.1 Sample vs Population Quantities . . . 15
- 3.2 Sampling from Distributions . . . 15
- 3.3 Monte Carlo Approximation . . . 16
- 3.4 Density Estimation . . . 17
  - 3.4.1 Histograms . . . 17
  - 3.4.2 Kernel Density Estimation . . . 17

4 Optimization **19**
- 4.1 Unconstrained Optimization . . . 19
  - 4.1.1 First-Order Optimality . . . 19
  - 4.1.2 Second-Order Conditions . . . 19
  - 4.1.3 Line Search Methods . . . 19
  - 4.1.4 Newton's Method . . . 20
  - 4.1.5 Trust Region Methods . . . 20
- 4.2 Constrained Optimization . . . 20
  - 4.2.1 Karush–Kuhn–Tucker (KKT) Conditions . . . 21
  - 4.2.2 Simplex Method . . . 21

4.2.3 Interior Point Methods . . . 21
4.2.4 Ellipsoid Method . . . 21
4.3 General Optimization Frameworks . . . 22
4.3.1 Penalty Methods . . . 22
4.3.2 Augmented Lagrangian . . . 22
4.3.3 Alternating Direction Method of Multipliers (ADMM) . . . 22

5 Least Squares **23**
5.1 Multivariate Probability Theory . . . 23
5.1.1 Dependence . . . 23
5.1.2 Covariance . . . 23
5.1.3 Correlation . . . 24
5.1.4 Vector Notation . . . 24
5.1.5 Bivariate Normal Distribution . . . 25
5.1.6 Multivariate Normal Distribution . . . 25
5.2 Ordinary Least Squares . . . 26
5.2.1 The Idea . . . 26
5.2.2 Fitting a Constant . . . 26
5.2.3 Weighted Constant Fit and Heteroscedasticity . . . 27
5.3 Simple Linear Regression . . . 27
5.4 Linear Regression with Basis Functions . . . 29
5.4.1 Normal Equations . . . 29
5.4.2 Componentwise Derivation . . . 30
5.4.3 Moore–Penrose Pseudoinverse . . . 31
5.5 Geometry of Least Squares . . . 31
5.6 Weighted Least Squares . . . 32
5.7 Regularization . . . 33
5.7.1 Ridge Regression . . . 33
5.7.2 Lasso Regression . . . 34
5.8 Least Squares Algorithm . . . 34
5.9 Implementation Notes . . . 34
5.10 Linear Combinations and Basis Expansions . . . 35

6 Principal Components Analysis **36**
6.1 Directions of Maximum Variance . . . 36
6.2 Constrained Optimization Derivation . . . 36
6.3 Second Principal Component . . . 38
6.4 Principal Components Analysis . . . 39
6.5 Low-Rank Approximation . . . 40
6.6 New Coordinate System . . . 40
6.7 Samples and the Sample Covariance Matrix . . . 41
6.8 Connection to Singular Value Decomposition . . . 41
6.9 Whitening . . . 42
6.10 Scree Plot and Explained Variance . . . 42
6.11 PCA Algorithm . . . 43

6.12 Implementation Notes . . . 44

7 Nearest Neighbors 45
7.1 Classification . . . 45
7.2 Nearest Neighbors . . . 45
7.3 k-Nearest Neighbors . . . 46
7.4 Bias-Variance Tradeoff . . . 46
7.5 Evaluating on a Grid . . . 46
7.6 Meaningful Distance . . . 47
7.7 Curse of Dimensionality . . . 47
7.7.1 Volume Concentration . . . 48
7.8 Implementation Notes . . . 48

8 Naive Bayes, LDA, and QDA 49
8.1 Naive Bayes Classifier . . . 49
8.1.1 Bayesian Classification . . . 49
8.1.2 Naive Independence Assumption . . . 49
8.1.3 Gaussian Naive Bayes . . . 50
8.1.4 Log-Domain Form . . . 50
8.1.5 Advantages and Limitations . . . 50
8.2 Discriminant Analysis . . . 50
8.2.1 Quadratic Discriminant Analysis (QDA) . . . 50
8.2.2 Binary QDA Decision Boundary . . . 51
8.3 Linear Discriminant Analysis (LDA) . . . 51
8.3.1 Linearization of the Decision Boundary . . . 51
8.3.2 Parameter Estimation . . . 52
8.3.3 Comparison: LDA vs QDA . . . 52
8.4 Algorithmic Summary . . . 53
8.5 Implementation Notes . . . 53

9 Cross Validation 56
9.1 Model Evaluation . . . 56
9.2 Train-Test Split and Complementary Subsets . . . 56
9.3 Repeated Random Subsampling . . . 57
9.4 Leave-One-Out Cross Validation (LOOCV) . . . 57
9.5 k-Fold Cross Validation . . . 57
9.6 Bias-Variance Tradeoff in Cross Validation . . . 58
9.7 Cross Validation Algorithm . . . 58
9.8 Implementation Notes . . . 58

10 Decision Trees 60
10.1 Tree Structures . . . 60
10.1.1 n-ary Trees . . . 60
10.2 Trees in Computation . . . 60
10.2.1 kd-Trees . . . 60

10.3 Decision Trees . . . 61
10.3.1 Recursive Partitioning . . . 61
10.3.2 Impurity Minimization . . . 61
10.3.3 Gini Impurity . . . 61
10.3.4 Example: Root Impurity . . . 61
10.3.5 Example: Split Impurity . . . 62
10.3.6 Weighted Impurity . . . 62
10.4 Regression Trees . . . 62
10.5 Greedy Tree Construction . . . 62
10.6 Properties of Decision Trees . . . 63
10.7 Regularization and Practical Considerations . . . 63
10.8 Implementation Notes . . . 63

11 Scikit-Learn Pipeline and Random Forests 65
11.1 Scikit-Learn Pipeline and Grid Search . . . 65
11.1.1 Pipelines . . . 65
11.1.2 Grid Search with Cross Validation . . . 65
11.2 Random Forests . . . 66
11.2.1 Random Trees . . . 66
11.2.2 Forest of Random Trees . . . 66
11.2.3 Bagging and Variance Reduction . . . 66
11.2.4 Connection to the Central Limit Theorem . . . 67
11.3 Feature Selection in Random Forests . . . 67
11.4 Assumptions and Limitations . . . 67
11.5 Divide and Conquer Perspective . . . 67
11.6 Random Forest Algorithm . . . 68
11.7 Implementation Notes . . . 68

12 Logistic Regression 70
12.1 Binary Classification and the Logistic Function . . . 70
12.2 Log-Odds and Linear Model . . . 70
12.3 Likelihood and Estimation . . . 71
12.3.1 Gradient and Optimization . . . 71
12.4 Prediction . . . 72
12.5 Decision Boundary . . . 72
12.6 Multiclass Logistic Regression . . . 72
12.7 Discriminative vs Generative Models . . . 72
12.8 Regularization . . . 73
12.8.1 L2 Regularization (Ridge) . . . 73
12.8.2 L1 Regularization (Lasso) . . . 73
12.9 Algorithm . . . 73
12.10Implementation Notes . . . 73

13 Support Vector Machines 75
13.1 Maximum Margin Classification . . . 75

13.2 Hard Margin SVM . . . 75
13.2.1 Margin Width . . . 75
13.2.2 Optimization Problem . . . 76
13.3 Support Vectors . . . 76
13.4 Soft Margin SVM . . . 76
13.4.1 Hinge Loss Formulation . . . 77
13.5 Kernel Methods . . . 77
13.5.1 Feature Transformation . . . 77
13.5.2 Kernel Trick . . . 77
13.5.3 Common Kernels . . . 77
13.6 Dual Formulation (Key Insight) . . . 78
13.7 Algorithmic Summary . . . 78
13.8 Properties and Comparison . . . 78
13.9 Implementation Notes . . . 78

14 k-Means Clustering 80
14.1 Clustering . . . 80
14.2 Types of Clustering Algorithms . . . 80
14.2.1 Flat (Partitioning) Methods . . . 80
14.2.2 Hierarchical Methods . . . 80
14.3 k-Means Clustering . . . 80
14.3.1 Optimization Problem . . . 80
14.3.2 Optimality of the Mean . . . 81
14.4 Algorithm . . . 81
14.5 Inertia . . . 82
14.6 Choosing the Number of Clusters . . . 82
14.6.1 Elbow Method . . . 82
14.7 Limitations . . . 83
14.7.1 Initialization Strategies . . . 83
14.8 Variants . . . 83
14.8.1 k-Medians . . . 83
14.9 Geometric Interpretation . . . 83
14.10 Implementation Notes . . . 83

15 Gaussian Mixture Models 85
15.1 Probabilistic Clustering . . . 85
15.2 Latent Variables . . . 85
15.3 Likelihood Function . . . 86
15.4 Expectation-Maximization (EM) Algorithm . . . 86
15.4.1 E-Step . . . 86
15.4.2 M-Step . . . 86
15.4.3 Algorithm . . . 87
15.5 Connection to k-Means . . . 87
15.6 Soft Clustering . . . 87
15.7 Properties and Limitations . . . 87

15.8 Model Selection . . . 87
15.9 Comparison with k-Means . . . 88
15.10Implementation Notes . . . 88
15.11Further Clustering Methods . . . 88

16 Spectral Embedding and Spectral Clustering 89
16.1 Spectral Methods . . . 89
16.2 Graphs . . . 89
16.3 Similarity Graphs . . . 89
16.4 Adjacency Matrix . . . 90
16.5 Degree Matrix and Graph Laplacian . . . 90
16.6 Spectral Embedding . . . 91
16.7 Spectral Clustering Algorithm . . . 91
16.8 Interpretation . . . 91
16.9 Properties of the Laplacian . . . 91
16.10Implementation Notes . . . 92

17 Laplacian Eigenmaps 93
17.1 Motivation . . . 93
17.2 Graph Construction . . . 93
17.3 Graph Laplacian . . . 93
17.4 Optimization Problem . . . 94
17.4.1 Matrix Form . . . 94
17.5 Constraints . . . 94
17.6 Solution via Eigenvalue Problem . . . 95
17.7 Interpretation . . . 95
17.8 Connection to Spectral Clustering . . . 95
17.9 Algorithm . . . 95
17.10Properties and Limitations . . . 95
17.11Implementation Notes . . . 96

18 Manifold Learning 97
18.1 Motivation . . . 97
18.2 Metric Multidimensional Scaling (MDS) . . . 97
18.2.1 Problem Formulation . . . 97
18.2.2 Classical MDS . . . 97
18.3 Isomap . . . 98
18.3.1 Idea . . . 98
18.3.2 Algorithm . . . 98
18.4 Locally Linear Embedding (LLE) . . . 98
18.4.1 Idea . . . 98
18.4.2 Embedding Step . . . 99
18.4.3 Solution . . . 99
18.5 Comparison of Methods . . . 99
18.6 Other Embedding Methods . . . 100

18.6.1 Random Projection . . . 100
18.6.2 PCA Projection . . . 100
18.6.3 Spectral Embedding . . . 100
18.6.4 t-SNE . . . 100
18.7 Implementation Notes . . . 100

19 SQLite Databases and SQL **101**
19.1 Overview . . . 101
19.2 Database Schema . . . 101
19.3 Basic SQL Queries . . . 101
19.3.1 Selecting Data . . . 101
19.3.2 Sorting . . . 102
19.4 Aggregation . . . 102
19.4.1 Grouping . . . 102
19.5 Joins . . . 102
19.5.1 Inner Join . . . 103
19.5.2 Multiple Joins . . . 103
19.6 Subqueries . . . 103
19.7 Common Table Expressions (CTEs) . . . 103
19.8 Relational Algebra Perspective . . . 104
19.9 Table Creation and Modification . . . 104
19.9.1 Creating Tables . . . 104
19.9.2 Inserting Data . . . 104
19.9.3 Updating Data . . . 104
19.9.4 Deleting Data . . . 104
19.10Transactions . . . 104
19.11Python Integration . . . 105

Chapter 1

Descriptive Statistics

1.1 Data Sets

A data set consisting of N scalar observations is denoted by

$$\{x_i\}_{i=1}^{N}, \quad x_i \in \mathbb{R}.$$

In data analysis, we typically summarize a data set through three main characteristics:

- Location (central tendency)
- Dispersion (variability)
- Shape (distributional features such as skewness and modality)

1.2 Measures of Location

Definition 1.2.1 (Mean)**.** The sample mean of a data set $\{x_i\}_{i=1}^{N}$ is

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i.$$

In computational settings (e.g., Python), indexing often begins at 0, in which case the mean is written as

$$\bar{x} = \frac{1}{N} \sum_{i=0}^{N-1} x_i.$$

Definition 1.2.2 (Median)**.** Let $\{x_{(i)}\}_{i=1}^{N}$ denote the sorted data such that $x_{(1)} \leq \cdots \leq x_{(N)}$. The sample median is defined as

$$\text{median} = \begin{cases} x_{(\frac{N+1}{2})}, & \text{if } N \text{ is odd,} \\ \frac{1}{2}\Big(x_{(\frac{N}{2})} + x_{(\frac{N}{2}+1)}\Big), & \text{if } N \text{ is even.} \end{cases}$$

Definition 1.2.3 (Mode)**.** The mode is the value(s) that occur with the highest frequency in the data set. A distribution may be unimodal or multimodal depending on the number of such values.

1.3 Measures of Dispersion

Definition 1.3.1 (Sample Variance)**.** The sample variance is defined as

$$s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2.$$

Definition 1.3.2 (Sample Standard Deviation)**.** The sample standard deviation is

$$s = \sqrt{s^2}.$$

Remark 1.3.1 (Degrees of Freedom)**.** The factor $N-1$ arises from estimating the population mean using $\bar{x}$. This reduces the effective degrees of freedom by one, leading to an unbiased estimator of the population variance.

Proposition 1.3.1 (Unbiasedness of Sample Variance)**.** *Let $X_1, \ldots, X_N \overset{iid}{\sim}$ with mean μ and variance σ^2. Then*

$$\mathbb{E}[s^2] = \sigma^2.$$

Proof. We write

$$\sum_{i=1}^{N}(X_i - \bar{X})^2 = \sum_{i=1}^{N}(X_i - \mu)^2 - N(\bar{X} - \mu)^2.$$

Taking expectations,

$$\mathbb{E}\left[\sum_{i=1}^{N}(X_i - \mu)^2\right] = N\sigma^2, \quad \mathbb{E}\left[N(\bar{X} - \mu)^2\right] = N \cdot \frac{\sigma^2}{N} = \sigma^2.$$

Thus,

$$\mathbb{E}\left[\sum_{i=1}^{N}(X_i - \bar{X})^2\right] = (N-1)\sigma^2.$$

Dividing by $N-1$ yields $\mathbb{E}[s^2] = \sigma^2$. □

1.4 Outliers

Definition 1.4.1 (Outlier)**.** An outlier is an observation that deviates significantly from the rest of the data.

Remark 1.4.1. Extreme values can heavily influence the mean and variance. For example, if a single observation diverges (e.g., $x_i \to \infty$), then

$$\bar{x} \to \infty, \quad s^2 \to \infty.$$

1.5 Robustness

Definition 1.5.1 (Robust Statistic)**.** A statistic is said to be robust if it is not unduly affected by small deviations from model assumptions or the presence of outliers.

Proposition 1.5.1. *The median is more robust to outliers than the mean.*

Proof. The mean depends linearly on all observations:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^{N} x_i,$$

so a single extreme value can arbitrarily shift $\bar{x}$.

In contrast, the median depends only on the ordering of the data. A single extreme value changes only the endpoints of the ordered sequence and does not affect the middle element(s), leaving the median unchanged unless the outlier crosses the median threshold. □

Definition 1.5.2 (Median Absolute Deviation (MAD))**.** The median absolute deviation is defined as

$$\text{MAD} = \text{median}\left(|x_i - \text{median}(x)|\right).$$

Remark 1.5.1. MAD provides a robust alternative to standard deviation, as it is based on medians rather than squared deviations, making it less sensitive to extreme values.

Chapter 2

Distributions

2.1 Probability Density Functions

Definition 2.1.1 (Probability Density Function)**.** A function $p : \mathbb{R} \to [0, \infty)$ is called a probability density function (PDF) if

$$\int_{-\infty}^{\infty} p(x)\,dx = 1.$$

Proposition 2.1.1. *For a continuous random variable X with density $p(x)$, the probability that X lies in an interval (a, b) is*

$$\mathbb{P}(a < X < b) = \int_a^b p(x)\,dx.$$

Example 2.1.1 (Uniform Distribution)**.** The uniform distribution on (a, b) has density

$$p(x) = \frac{\mathbb{1}_{(a,b)}(x)}{b-a},$$

where $\mathbb{1}_{(a,b)}(x)$ is the indicator function.

Example 2.1.2 (Gaussian Distribution)**.** The Gaussian (normal) distribution with mean μ and variance σ^2 has density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

2.2 Cumulative Distribution Function

Definition 2.2.1 (Cumulative Distribution Function)**.** The cumulative distribution function (CDF) of a random variable X is

$$F(x) = \mathbb{P}(X \leq x) = \int_{-\infty}^{x} p(t)\,dt.$$

Proposition 2.2.1. *The CDF $F(x)$ satisfies:*

- *$F(x)$ is non-decreasing,*
- *$\lim_{x\to-\infty} F(x) = 0$ and $\lim_{x\to\infty} F(x) = 1$,*
- *If $p(x)$ is continuous, then $F'(x) = p(x)$.*

2.3 Expectation and Moments

Definition 2.3.1 (Expectation)**.** The expectation of a random variable X with density $p(x)$ is

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x\, p(x)\, dx.$$

Definition 2.3.2 (Expectation of a Function)**.** For any measurable function f,

$$\mathbb{E}[f(X)] = \int_{-\infty}^{\infty} f(x)\, p(x)\, dx.$$

Definition 2.3.3 (Moments)**.** The k-th moment of X is

$$\mathbb{E}[X^k].$$

The k-th central moment is

$$\mathbb{E}[(X-\mu)^k], \quad \mu = \mathbb{E}[X].$$

Definition 2.3.4 (Variance)**.** The variance of X is

$$\mathrm{Var}(X) = \mathbb{E}[(X-\mu)^2].$$

Proposition 2.3.1 (Variance Identity)**.**

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Proof. Expanding,

$$\mathrm{Var}(X) = \mathbb{E}[(X-\mu)^2] = \mathbb{E}[X^2 - 2\mu X + \mu^2] = \mathbb{E}[X^2] - 2\mu\mathbb{E}[X] + \mu^2.$$

Since $\mu = \mathbb{E}[X]$, this simplifies to

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - \mu^2.$$

□

Definition 2.3.5 (Standard Deviation)**.** The standard deviation is

$$\sigma = \sqrt{\mathrm{Var}(X)}.$$

Definition 2.3.6 (Standardized Moments)**.** The k-th standardized moment is

$$\mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^k\right].$$

Definition 2.3.7 (Skewness and Kurtosis)**.**

- Skewness: $\mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^3\right]$
- Kurtosis: $\mathbb{E}\left[\left(\frac{X-\mu}{\sigma}\right)^4\right]$

2.4 Python Introduction

2.4.1 Core Language Concepts

- **Tuple**: immutable ordered collection

```
x = (1, 2, 3)
```

- **List**: mutable ordered collection

```
x = [1, 2, 3]
```

- **Function**

```
def f(x):
    return x**2
```

- **Lambda**

```
f = lambda x: x**2
```

- **For loop**

```
for i in range(5):
    print(i)
```

- **Map**

```
list(map(lambda x: x**2, [1,2,3]))
```

- **Importing libraries**

```
import numpy as np
import matplotlib.pyplot as plt
```

2.4.2 Pandas Introduction

Pandas is a library for structured data manipulation.

```
import pandas as pd

df = pd.DataFrame({
    "x": [1,2,3],
    "y": [4,5,6]
})

df.mean()
df.describe()
```

2.4.3 Seaborn Introduction

Seaborn is a visualization library built on top of matplotlib.

```
import seaborn as sns

sns.histplot(df["x"])
sns.scatterplot(x="x", y="y", data=df)
```

Chapter 3

Samples and Kernels

3.1 Sample vs Population Quantities

Proposition 3.1.1 (Sample Estimates vs Population Quantities)**.** *Let* $\{x_i\}_{i=1}^N$ *be a sample from a random variable* $X \sim p(\cdot)$*. Then*

$$\bar{x} = \frac{1}{N}\sum_{i=1}^{N} x_i = \langle x_i \rangle_{i=1}^{N} \quad \longleftrightarrow \quad \mu = \mathbb{E}[X] = \int x\, p(x)\, dx,$$

$$s^2 = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2 \quad \longleftrightarrow \quad \mathrm{Var}(X) = \int (x-\mu)^2 p(x)\, dx.$$

Remark 3.1.1. Sample statistics approximate population quantities. The quality of approximation improves with increasing N.

3.2 Sampling from Distributions

Proposition 3.2.1 (Uniform Scaling)**.** *If* $U \sim \mathrm{Uniform}(0,1)$*, then*

$$X = a + (b-a)U \sim \mathrm{Uniform}(a,b).$$

Proof. For $x \in (a,b)$,

$$\mathbb{P}(X \le x) = \mathbb{P}\left(U \le \frac{x-a}{b-a}\right) = \frac{x-a}{b-a},$$

which is the CDF of $\mathrm{Uniform}(a,b)$. □

Proposition 3.2.2 (Inverse Transform Sampling)**.** *Let* $U \sim \mathrm{Uniform}(0,1)$ *and let* F *be a strictly increasing CDF. Then*

$$X = F^{-1}(U)$$

has distribution with CDF F*.*

Proof. For any x,

$$\mathbb{P}(X \leq x) = \mathbb{P}(F^{-1}(U) \leq x) = \mathbb{P}(U \leq F(x)) = F(x).$$

□

Definition 3.2.1 (Rejection Sampling)**.** To sample from a target density $p(x)$:

1. Choose proposal $q(x)$ and constant M such that $p(x) \leq Mq(x)$.
2. Sample $X \sim q(x)$ and $U \sim \text{Uniform}(0, 1)$.
3. Accept X if $U \leq \frac{p(X)}{Mq(X)}$.

Remark 3.2.1. Rejection sampling extends naturally to $\mathbb{R}^d$, but efficiency deteriorates in high dimensions due to low acceptance rates.

3.3 Monte Carlo Approximation

Proposition 3.3.1 (Law of Large Numbers Approximation)**.** *Let* $x_i \overset{iid}{\sim} p(\cdot)$*. Then*

$$\mathbb{E}[X] = \int x\, p(x)\, dx \approx \frac{1}{N} \sum_{i=1}^{N} x_i,$$

$$\mathbb{E}[f(X)] = \int f(x)\, p(x)\, dx \approx \frac{1}{N} \sum_{i=1}^{N} f(x_i).$$

Proposition 3.3.2 (Variance Approximation)**.**

$$\text{Var}(X) = \mathbb{E}[(X - \mu)^2] \approx \frac{1}{N} \sum_{i=1}^{N} (x_i - \mu)^2.$$

Remark 3.3.1 (Bessel Correction)**.** The estimator

$$s^2 = \frac{1}{N-1} \sum_{i=1}^{N} (x_i - \bar{x})^2$$

uses $N - 1$ degrees of freedom because

$$\sum_{i=1}^{N} (x_i - \bar{x}) = 0,$$

implying only $N - 1$ independent deviations.

3.4 Density Estimation

3.4.1 Histograms

Definition 3.4.1 (Histogram Estimator)**.** Let bin width be h and bin origin x_0. Then

$$\text{Hist}(x) = \frac{1}{Nh} \sum_{i=1}^{N} \mathbb{1}_{\text{bin}(x_i;x_0,h)}(x).$$

Remark 3.4.1. Histograms depend heavily on the choice of bin width h and alignment x_0, leading to discontinuous density estimates.

3.4.2 Kernel Density Estimation

Definition 3.4.2 (Kernel Density Estimator)**.** Let K be a kernel function and $h > 0$ a bandwidth. Then

$$\widehat{p}(x) = \frac{1}{N} \sum_{i=1}^{N} K_h(x - x_i) = \frac{1}{Nh} \sum_{i=1}^{N} K\left(\frac{x - x_i}{h}\right).$$

Remark 3.4.2. The bandwidth h controls the bias-variance tradeoff:

- Small h: low bias, high variance (overfitting)
- Large h: high bias, low variance (oversmoothing)

Definition 3.4.3 (Kernel Function)**.** A kernel K satisfies:

- $K(x) \geq 0$
- $\int K(x)\, dx = 1$
- Often symmetric: $K(x) = K(-x)$

Example 3.4.1 (Common Kernels)**.**

- Uniform: $K(x) = \frac{1}{2}\mathbb{1}_{|x|\leq 1}$
- Triangular: $K(x) = (1 - |x|)\mathbb{1}_{|x|\leq 1}$
- Gaussian: $K(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$
- Epanechnikov: $K(x) = \frac{3}{4}(1 - x^2)\mathbb{1}_{|x|\leq 1}$

Remark 3.4.3 (Support)**.** Kernels may have:

- Finite support (e.g., uniform, Epanechnikov)
- Infinite support (e.g., Gaussian)

Remark 3.4.4 (Numerical Considerations)**.** In practice:

- Gaussian kernels are widely used due to smoothness and differentiability
- Finite-support kernels are computationally cheaper
- Bandwidth selection dominates kernel choice in importance

Chapter 4

Optimization

4.1 Unconstrained Optimization

We consider problems of the form

$$\min_{x \in \mathbb{R}^d} f(x),$$

where f is typically assumed to be differentiable.

4.1.1 First-Order Optimality

Theorem 4.1.1 (First-Order Condition)**.** *If f is differentiable and $x^\star$ is a local minimizer, then*

$$\nabla f(x^\star) = 0.$$

4.1.2 Second-Order Conditions

Theorem 4.1.2 (Second-Order Condition)**.** *If f is twice differentiable and $x^\star$ is a local minimizer, then*

$$\nabla f(x^\star) = 0, \quad \nabla^2 f(x^\star) \succeq 0.$$

If $\nabla^2 f(x^\star) \succ 0$, then $x^\star$ is a strict local minimum.

4.1.3 Line Search Methods

Iterative update:

$$x_{k+1} = x_k + \alpha_k d_k,$$

where d_k is a descent direction.

```
# Gradient Descent with Line Search
x = x0
for k in range(max_iter):
    grad = grad_f(x)
    d = -grad
```

```
    alpha = line_search(f, x, d)
    x = x + alpha * d
```

Remark 4.1.1. Common choices for d_k include $-\nabla f(x_k)$ (gradient descent) and quasi-Newton directions.

4.1.4 Newton's Method

Using second-order information:

$$x_{k+1} = x_k - \nabla^2 f(x_k)^{-1} \nabla f(x_k).$$

```
# Newton's Method
x = x0
for k in range(max_iter):
    g = grad_f(x)
    H = hess_f(x)
    x = x - np.linalg.solve(H, g)
```

Remark 4.1.2. Newton's method has quadratic convergence near the optimum but requires solving a linear system at each step.

4.1.5 Trust Region Methods

Solve subproblem:

$$\min_{\|p\| \le \Delta} \quad m_k(p) = f(x_k) + \langle \nabla f(x_k), p \rangle + \frac{1}{2} \left\langle p, \nabla^2 f(x_k) p \right\rangle.$$

```
# Trust Region (conceptual)
for k in range(max_iter):
    p = solve_trust_region_subproblem(grad, H, Delta)
    if actual_reduction / predicted_reduction > eta:
        x = x + p
        Delta = increase(Delta)
    else:
        Delta = decrease(Delta)
```

4.2 Constrained Optimization

General problem:

$$\min_{x \in \mathbb{R}^d} f(x) \quad \text{s.t.} \quad g_i(x) \le 0,\ h_j(x) = 0.$$

4.2.1 Karush–Kuhn–Tucker (KKT) Conditions

Theorem 4.2.1 (KKT Conditions)**.** *If $x^\star$ is optimal and regularity conditions hold, then there exist multipliers $\lambda_i \geq 0$, ν_j such that*

$$\nabla f(x^\star) + \sum_i \lambda_i \nabla g_i(x^\star) + \sum_j \nu_j \nabla h_j(x^\star) = 0,$$

$$g_i(x^\star) \leq 0, \quad h_j(x^\star) = 0,$$
$$\lambda_i g_i(x^\star) = 0.$$

4.2.2 Simplex Method

Used for linear programs:

$$\min c^\top x \quad \text{s.t.} \quad Ax = b,\ x \geq 0.$$

```
# Conceptual Simplex
initialize basic feasible solution
while optimality not reached:
    choose entering variable
    choose leaving variable
    pivot
```

4.2.3 Interior Point Methods

Solve barrier problem:

$$\min_x \ f(x) - \mu \sum_i \log(-g_i(x)).$$

```
# Interior Point (Barrier)
x = x0
for t in schedule:
    minimize f(x) - mu * sum(log(-g(x)))
    decrease mu
```

4.2.4 Ellipsoid Method

Iteratively shrink ellipsoid containing feasible region.

```
# Ellipsoid (conceptual)
initialize ellipsoid E0
for k in range(max_iter):
    if center feasible:
        update best solution
    else:
        cut ellipsoid using separating hyperplane
```

4.3 General Optimization Frameworks

4.3.1 Penalty Methods

Convert constrained problem into unconstrained:

$$\min_x \ f(x) + \rho \sum_i \max(0, g_i(x))^2.$$

Remark 4.3.1. As $\rho \to \infty$, solutions approach feasibility.

4.3.2 Augmented Lagrangian

$$\mathcal{L}_\rho(x, \lambda) = f(x) + \lambda^\top g(x) + \frac{\rho}{2} \|g(x)\|^2 .$$

```
# Augmented Lagrangian
for k in range(max_iter):
    x = minimize L_rho(x, lambda)
    lambda = lambda + rho * g(x)
```

4.3.3 Alternating Direction Method of Multipliers (ADMM)

Split problem:

$$\min f(x) + g(z) \quad \text{s.t.} \quad Ax + Bz = c.$$

Iterations:

$$x^{k+1} = \arg\min_x \mathcal{L}_\rho(x, z^k, y^k),$$

$$z^{k+1} = \arg\min_z \mathcal{L}_\rho(x^{k+1}, z, y^k),$$

$$y^{k+1} = y^k + \rho(Ax^{k+1} + Bz^{k+1} - c).$$

```
# ADMM
for k in range(max_iter):
    x = argmin_x L(x, z, y)
    z = argmin_z L(x, z, y)
    y = y + rho * (A@x + B@z - c)
```

Remark 4.3.2. ADMM is especially useful for large-scale problems and distributed optimization.

Chapter 5

Least Squares

5.1 Multivariate Probability Theory

5.1.1 Dependence

Let $X, Y \in \mathbb{R}$ be random variables. They are independent if

$$p(x, y) = p_X(x)p_Y(y).$$

Equivalently, for events A and B,

$$\mathbb{P}(X \in A, Y \in B) = \mathbb{P}(X \in A)\mathbb{P}(Y \in B).$$

If

$$p(x, y) \neq p_X(x)p_Y(y),$$

then X and Y are dependent.

5.1.2 Covariance

Definition 5.1.1 (Covariance)**.** The covariance between X and Y is

$$\mathrm{Cov}(X, Y) = \mathbb{E}\left[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])\right].$$

Equivalent notation includes $C_{X,Y}$ and $\sigma(X, Y)$.

Proposition 5.1.1 (Covariance Identity)**.**

$$\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

Proof. Expanding the definition,

$$\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mu_X)(Y - \mu_Y)].$$

Thus,

$$\mathrm{Cov}(X,Y) = \mathbb{E}[XY - X\mu_Y - Y\mu_X + \mu_X\mu_Y].$$

Using linearity of expectation,

$$\mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mu_Y\mathbb{E}[X] - \mu_X\mathbb{E}[Y] + \mu_X\mu_Y.$$

Since $\mu_X = \mathbb{E}[X]$ and $\mu_Y = \mathbb{E}[Y]$,

$$\mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mu_X\mu_Y.$$

□

Definition 5.1.2 (Sample Covariance)**.** Given paired data $\{(x_i, y_i)\}_{i=1}^N$, the sample covariance is

$$C_{xy} = \frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y}).$$

5.1.3 Correlation

Definition 5.1.3 (Correlation)**.** The correlation between X and Y is

$$\mathrm{Corr}(X,Y) = \frac{\mathrm{Cov}(X,Y)}{\sigma_X\sigma_Y},$$

where $\sigma_X = \sqrt{\mathrm{Var}(X)}$ and $\sigma_Y = \sqrt{\mathrm{Var}(Y)}$.

Remark 5.1.1. Correlation is a standardized covariance. It satisfies

$$-1 \le \mathrm{Corr}(X,Y) \le 1.$$

Values near 1 indicate strong positive linear association, values near -1 indicate strong negative linear association, and values near 0 indicate weak linear association.

5.1.4 Vector Notation

Let

$$V = \begin{pmatrix} X \\ Y \end{pmatrix}.$$

Then

$$\mathbb{E}[V] = \begin{pmatrix} \mathbb{E}[X] \\ \mathbb{E}[Y] \end{pmatrix} = \begin{pmatrix} \mu_X \\ \mu_Y \end{pmatrix}.$$

Definition 5.1.4 (Covariance Matrix)**.** The covariance matrix of V is

$$\Sigma_V = \mathbb{E}\left[(V - \mathbb{E}[V])(V - \mathbb{E}[V])^T\right].$$

For $V = (X,Y)^T$,

$$\Sigma_V = \begin{pmatrix} \sigma_X^2 & \mathrm{Cov}(X,Y) \\ \mathrm{Cov}(Y,X) & \sigma_Y^2 \end{pmatrix}.$$

Remark 5.1.2. Since $\mathrm{Cov}(X, Y) = \mathrm{Cov}(Y, X)$, covariance matrices are symmetric.

More generally, for a random vector $X \in \mathbb{R}^d$,

$$\Sigma = \mathbb{E}\left[(X - \mu)(X - \mu)^T\right],$$

where $\mu = \mathbb{E}[X]$.

5.1.5 Bivariate Normal Distribution

If X and Y are independent normal random variables with means μ_X, μ_Y and variances σ_X^2, σ_Y^2, then their joint density is

$$p(x, y) = \frac{1}{2\pi\sigma_X\sigma_Y} \exp\left[-\frac{(x - \mu_X)^2}{2\sigma_X^2} - \frac{(y - \mu_Y)^2}{2\sigma_Y^2}\right].$$

In vector notation, for $x \in \mathbb{R}^2$,

$$p(x) = \frac{1}{2\pi|\Sigma|^{1/2}} \exp\left[-\frac{1}{2}(x - \mu)^T\Sigma^{-1}(x - \mu)\right].$$

Here $|\Sigma| = \det(\Sigma)$.

If

$$\Sigma = \begin{pmatrix} \sigma_X^2 & 0 \\ 0 & \sigma_Y^2 \end{pmatrix},$$

then X and Y are uncorrelated.

Remark 5.1.3. For jointly Gaussian random variables, uncorrelatedness implies independence. This is not true for arbitrary distributions.

5.1.6 Multivariate Normal Distribution

Definition 5.1.5 (Multivariate Normal Distribution)**.** For $x, \mu \in \mathbb{R}^k$ and positive definite covariance matrix $\Sigma \in \mathbb{R}^{k\times k}$,

$$p(x) = \frac{1}{\sqrt{(2\pi)^k|\Sigma|}} \exp\left[-\frac{1}{2}(x - \mu)^T\Sigma^{-1}(x - \mu)\right].$$

Equivalently,

$$p(x) = \frac{1}{\sqrt{|2\pi\Sigma|}} \exp\left[-\frac{1}{2}(x - \mu)^T\Sigma^{-1}(x - \mu)\right].$$

5.2 Ordinary Least Squares

5.2.1 The Idea

Suppose we have a training set

$$\{(x_i, y_i)\}_{i=1}^N.$$

We want to fit a parameterized function

$$f(x; \theta),$$

where θ may contain one or more parameters.

The residual for observation i is

$$r_i(\theta) = y_i - f(x_i; \theta).$$

Ordinary least squares chooses parameters by minimizing the sum of squared residuals:

$$\hat{\theta} = \arg\min_{\theta} \sum_{i=1}^{N} \left[y_i - f(x_i; \theta)\right]^2.$$

5.2.2 Fitting a Constant

Consider the constant model

$$f(x; \mu) = \mu.$$

The cost function is

$$C(\mu) = \sum_{i=1}^{N} (y_i - \mu)^2.$$

Proposition 5.2.1 (Least Squares Estimate of a Constant)**.** *The least squares estimate of μ is the sample mean:*

$$\hat{\mu} = \frac{1}{N} \sum_{i=1}^{N} y_i.$$

Proof. Differentiate the cost:

$$\frac{dC}{d\mu} = \frac{d}{d\mu} \sum_{i=1}^{N} (y_i - \mu)^2 = -2 \sum_{i=1}^{N} (y_i - \mu).$$

At the optimum,

$$-2 \sum_{i=1}^{N} (y_i - \hat{\mu}) = 0.$$

Therefore,

$$\sum_{i=1}^{N} y_i - N\hat{\mu} = 0,$$

and hence

$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} y_i.$$

□

5.2.3 Weighted Constant Fit and Heteroscedasticity

Suppose observations have different variances σ_i^2. Define weights

$$w_i = \frac{1}{\sigma_i^2}.$$

The weighted least squares cost is

$$C(\mu) = \sum_{i=1}^{N} w_i (y_i - \mu)^2.$$

Proposition 5.2.2 (Weighted Average)**.** *The weighted least squares estimate is*

$$\hat{\mu} = \frac{\sum_{i=1}^{N} w_i y_i}{\sum_{i=1}^{N} w_i}.$$

Proof. Differentiate:

$$\frac{dC}{d\mu} = -2\sum_{i=1}^{N} w_i (y_i - \mu).$$

At the optimum,

$$\sum_{i=1}^{N} w_i (y_i - \hat{\mu}) = 0.$$

Thus,

$$\sum_{i=1}^{N} w_i y_i - \hat{\mu}\sum_{i=1}^{N} w_i = 0.$$

Solving for $\hat{\mu}$ gives

$$\hat{\mu} = \frac{\sum_{i=1}^{N} w_i y_i}{\sum_{i=1}^{N} w_i}.$$

□

5.3 Simple Linear Regression

Consider the linear model

$$f(x;\theta) = a + bx,$$

where

$$\theta = \begin{pmatrix} a \\ b \end{pmatrix}.$$

The least squares estimate solves

$$\hat{\theta} = \arg\min_{a,b} \sum_{i=1}^{N} \left[y_i - (a + bx_i) \right]^2 .$$

The cost is quadratic in a and b, so the first-order conditions form a linear system.

Proposition 5.3.1 (Simple Linear Regression Coefficients)**.** *The least squares estimates are*

$$\hat{b} = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{N}(x_i - \bar{x})^2},$$

and

$$\hat{a} = \bar{y} - \hat{b}\bar{x}.$$

Proof. The cost is

$$C(a, b) = \sum_{i=1}^{N}(y_i - a - bx_i)^2.$$

The first-order conditions are

$$\frac{\partial C}{\partial a} = -2\sum_{i=1}^{N}(y_i - a - bx_i) = 0,$$

and

$$\frac{\partial C}{\partial b} = -2\sum_{i=1}^{N} x_i(y_i - a - bx_i) = 0.$$

The first equation gives

$$\sum_{i=1}^{N} y_i = Na + b\sum_{i=1}^{N} x_i.$$

Dividing by N,

$$\bar{y} = a + b\bar{x},$$

so

$$a = \bar{y} - b\bar{x}.$$

Substituting into the second first-order condition yields

$$\sum_{i=1}^{N} x_i \left[y_i - \bar{y} - b(x_i - \bar{x}) \right] = 0.$$

Equivalently,

$$\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y}) = b\sum_{i=1}^{N}(x_i - \bar{x})^2.$$

Thus,

$$\hat{b} = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{N}(x_i - \bar{x})^2},$$

and

$$\hat{a} = \bar{y} - \hat{b}\bar{x}.$$

□

5.4 Linear Regression with Basis Functions

Let $\phi_1, \ldots, \phi_K$ be known basis functions. A linear regression model is

$$f(x; \beta) = \sum_{k=1}^{K} \beta_k \phi_k(x).$$

Equivalently,

$$f(x; \beta) = \beta^T \phi(x),$$

where

$$\beta = (\beta_1, \ldots, \beta_K)^T, \quad \phi(x) = (\phi_1(x), \ldots, \phi_K(x))^T.$$

The cost function is

$$C(\beta) = \sum_{i=1}^{N} \left[y_i - \sum_{k=1}^{K} \beta_k \phi_k(x_i) \right]^2.$$

Define the design matrix $X \in \mathbb{R}^{N \times K}$ by

$$X_{ik} = \phi_k(x_i).$$

Then the model predictions are

$$\hat{y} = X\beta,$$

and the cost becomes

$$C(\beta) = \|y - X\beta\|_2^2.$$

5.4.1 Normal Equations

Theorem 5.4.1 (Normal Equations)**.** *The least squares solution satisfies*

$$X^T X \hat{\beta} = X^T y.$$

If $X^T X$ is invertible, then

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

Proof. The cost is

$$C(\beta) = (y - X\beta)^T (y - X\beta).$$

Expanding,

$$C(\beta) = y^T y - 2\beta^T X^T y + \beta^T X^T X \beta.$$

Taking the gradient with respect to β,

$$\nabla_\beta C(\beta) = -2X^T y + 2X^T X \beta.$$

At the optimum,

$$-2X^T y + 2X^T X \hat{\beta} = 0.$$

Therefore,

$$X^T X\hat{\beta} = X^T y.$$

If $X^T X$ is invertible,

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

□

Remark 5.4.1 (Kronecker Delta)**.** The Kronecker delta is

$$\delta_{kl} = \begin{cases} 1, & k = l, \\ 0, & k \neq l. \end{cases}$$

It satisfies

$$\sum_l \delta_{kl} a_l = a_k.$$

This is the componentwise analogue of

$$Ia = a.$$

5.4.2 Componentwise Derivation

Starting from

$$C(\beta) = \sum_{i=1}^{N} \left[y_i - \sum_{k=1}^{K} X_{ik}\beta_k \right]^2,$$

differentiate with respect to β_l:

$$\frac{\partial C}{\partial \beta_l} = -2 \sum_{i=1}^{N} \left[y_i - \sum_{k=1}^{K} X_{ik}\beta_k \right] X_{il}.$$

At the optimum,

$$\sum_{i=1}^{N} \left[y_i - \sum_{k=1}^{K} X_{ik}\hat{\beta}_k \right] X_{il} = 0.$$

Therefore,

$$\sum_{i=1}^{N} X_{il} y_i = \sum_{k=1}^{K} \left(\sum_{i=1}^{N} X_{il} X_{ik} \right) \hat{\beta}_k.$$

This is precisely

$$X^T y = X^T X\hat{\beta}.$$

5.4.3 Moore–Penrose Pseudoinverse

When X^TX is invertible,

$$X^+ = (X^TX)^{-1}X^T,$$

so

$$\hat{\beta} = X^+y.$$

More generally, the Moore–Penrose pseudoinverse can be defined using the singular value decomposition. If

$$X = U\Sigma V^T,$$

then

$$X^+ = V\Sigma^+U^T,$$

where Σ^+ is formed by taking reciprocals of the nonzero singular values.

5.5 Geometry of Least Squares

Least squares projects y onto the column space of X.

The fitted values are

$$\hat{y} = X\hat{\beta}.$$

Using the closed form solution,

$$\hat{y} = X(X^TX)^{-1}X^Ty.$$

Definition 5.5.1 (Hat Matrix)**.** The hat matrix is

$$H = X(X^TX)^{-1}X^T.$$

Thus,

$$\hat{y} = Hy.$$

Proposition 5.5.1 (Projection Properties)**.** *The hat matrix satisfies*

$$H^T = H,$$

and

$$H^2 = H.$$

Proof. First,

$$H^T = \left[X(X^TX)^{-1}X^T\right]^T = X(X^TX)^{-1}X^T = H,$$

since X^TX is symmetric.

Next,

$$H^2 = X(X^TX)^{-1}X^TX(X^TX)^{-1}X^T.$$

Since X^TX appears in the middle,

$$H^2 = X(X^TX)^{-1}(X^TX)(X^TX)^{-1}X^T = X(X^TX)^{-1}X^T = H.$$

□

Remark 5.5.1. The residual vector is

$$r = y - \hat{y} = y - X\hat{\beta}.$$

The normal equations imply

$$X^Tr = 0.$$

Thus, the residuals are orthogonal to every column of the design matrix.

5.6 Weighted Least Squares

For heteroscedastic errors, use

$$C(\beta) = \sum_{i=1}^{N} w_i \left[y_i - \sum_{k=1}^{K} X_{ik}\beta_k \right]^2.$$

Let W be diagonal with

$$W_{ij} = w_i\delta_{ij}.$$

Then

$$C(\beta) = (y - X\beta)^TW(y - X\beta).$$

Theorem 5.6.1 (Weighted Least Squares)**.** *The weighted least squares estimator satisfies*

$$X^TWX\hat{\beta} = X^TWy.$$

If X^TWX is invertible, then

$$\hat{\beta} = (X^TWX)^{-1}X^TWy.$$

Proof. Expand the weighted cost:

$$C(\beta) = (y - X\beta)^TW(y - X\beta).$$

Taking the gradient,

$$\nabla_\beta C(\beta) = -2X^TWy + 2X^TWX\beta.$$

Setting the gradient equal to zero gives

$$X^TWX\hat{\beta} = X^TWy.$$

Solving gives

$$\hat{\beta} = (X^TWX)^{-1}X^TWy.$$

□

Remark 5.6.1. If errors are uncorrelated but have variances σ_i^2, then the optimal weights are

$$w_i = \frac{1}{\sigma_i^2}.$$

If errors are correlated with covariance matrix C, then the natural weight matrix is

$$W = C^{-1}.$$

5.7 Regularization

Least squares can become unstable when predictors are highly correlated or when $X^T X$ is nearly singular. Regularization stabilizes estimation by penalizing large coefficients.

5.7.1 Ridge Regression

Definition 5.7.1 (Ridge Regression)**.** Ridge regression solves

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2,$$

where $\lambda \geq 0$.

Theorem 5.7.1 (Ridge Solution)**.** *The ridge regression estimator is*

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

Proof. The objective is

$$C(\beta) = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta.$$

Taking the gradient,

$$\nabla_\beta C(\beta) = -2X^T y + 2X^T X\beta + 2\lambda\beta.$$

Setting the gradient equal to zero,

$$(X^T X + \lambda I)\hat{\beta} = X^T y.$$

Thus,

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y.$$

□

More generally, one may use a matrix penalty:

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\Gamma\beta\|_2^2.$$

The solution satisfies

$$(X^T X + \lambda \Gamma^T \Gamma)\hat{\beta} = X^T y.$$

5.7.2 Lasso Regression

Definition 5.7.2 (Lasso Regression)**.** Lasso regression solves

$$\hat{\beta} = \arg\min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1 .$$

Remark 5.7.1. The L_1 penalty encourages sparsity, meaning some coefficients are exactly zero. Unlike ridge regression, lasso does not generally have a simple closed form solution.

5.8 Least Squares Algorithm

Algorithm 1 Ordinary Least Squares via Normal Equations

Require: Design matrix $X \in \mathbb{R}^{N \times K}$, response vector $y \in \mathbb{R}^N$
Ensure: Coefficient estimate $\hat{\beta}$
1: Compute $A \leftarrow X^T X$
2: Compute $b \leftarrow X^T y$
3: Solve $A\hat{\beta} = b$
4: **return** $\hat{\beta}$

Algorithm 2 Weighted Least Squares

Require: Design matrix X, response vector y, weights $w_1, \ldots, w_N$
Ensure: Weighted least squares estimate $\hat{\beta}$
1: Form diagonal matrix W with $W_{ii} = w_i$
2: Compute $A \leftarrow X^T W X$
3: Compute $b \leftarrow X^T W y$
4: Solve $A\hat{\beta} = b$
5: **return** $\hat{\beta}$

5.9 Implementation Notes

```python
import numpy as np

def ordinary_least_squares(X, y):
    """
    Computes beta_hat = (X^T X)^(-1) X^T y.
    Uses solve rather than explicitly forming the inverse.
    """
    A = X.T @ X
    b = X.T @ y
    beta_hat = np.linalg.solve(A, b)
    return beta_hat
```

```python
def weighted_least_squares(X, y, w):
    """
    Weighted least squares with diagonal weights w.
    """
    W = np.diag(w)
    A = X.T @ W @ X
    b = X.T @ W @ y
    beta_hat = np.linalg.solve(A, b)
    return beta_hat
```

```python
def ridge_regression(X, y, lam):
    """
    Ridge regression:
    beta_hat = (X^T X + lambda I)^(-1) X^T y.
    """
    K = X.shape[1]
    A = X.T @ X + lam * np.eye(K)
    b = X.T @ y
    beta_hat = np.linalg.solve(A, b)
    return beta_hat
```

5.10 Linear Combinations and Basis Expansions

Linear regression is fundamentally about representing data as linear combinations of known basis functions:

$$f(x;\beta) = \sum_{k=1}^{K} \beta_k \phi_k(x).$$

Examples include:

- Polynomial regression
- Fourier series
- Discrete cosine transforms
- Spherical harmonics
- Image bases
- Shape bases

Remark 5.10.1. The term "linear regression" means linear in the parameters β, not necessarily linear in the input x. For example,

$$f(x;\beta) = \beta_0 + \beta_1 x + \beta_2 x^2$$

is still a linear regression model because it is linear in β.

Chapter 6

Principal Components Analysis

6.1 Directions of Maximum Variance

Let $X \in \mathbb{R}^N$ be a random vector with

$$\mathbb{E}[X] = 0$$

and covariance matrix

$$C = \mathbb{E}[XX^T].$$

For any direction $a \in \mathbb{R}^N$, the scalar projection of X onto a is

$$a^T X.$$

The variance of this projection is

$$\mathrm{Var}(a^T X) = \mathbb{E}[(a^T X)^2].$$

Since

$$(a^T X)^2 = a^T X X^T a,$$

we have

$$\mathrm{Var}(a^T X) = \mathbb{E}[a^T X X^T a] = a^T \mathbb{E}[XX^T] a = a^T C a.$$

Therefore, the direction of maximum variance solves

$$\max_{a \in \mathbb{R}^N} a^T C a \quad \text{s.t.} \quad a^T a = 1.$$

6.2 Constrained Optimization Derivation

Define the Lagrangian

$$\mathcal{L}(a, \lambda) = a^T C a - \lambda(a^T a - 1).$$

The constrained maximizer satisfies

$$\frac{\partial \mathcal{L}}{\partial \lambda} = 0, \quad \nabla_a \mathcal{L} = 0.$$

The first condition gives

$$a^T a = 1.$$

For the second condition, write

$$a^T C a = \sum_{i=1}^{N} \sum_{j=1}^{N} a_i C_{ij} a_j.$$

Differentiating with respect to a_k,

$$\frac{\partial}{\partial a_k} \left(\sum_{i=1}^{N} \sum_{j=1}^{N} a_i C_{ij} a_j \right) = \sum_{i=1}^{N} \sum_{j=1}^{N} \frac{\partial a_i}{\partial a_k} C_{ij} a_j + \sum_{i=1}^{N} \sum_{j=1}^{N} a_i C_{ij} \frac{\partial a_j}{\partial a_k}.$$

Using

$$\frac{\partial a_i}{\partial a_k} = \delta_{ik},$$

we get

$$\sum_{j=1}^{N} C_{kj} a_j + \sum_{i=1}^{N} a_i C_{ik}.$$

In matrix notation, this is

$$Ca + C^T a.$$

Since covariance matrices are symmetric, $C = C^T$, so

$$\nabla_a (a^T C a) = 2Ca.$$

Also,

$$\nabla_a \{\lambda (a^T a - 1)\} = 2\lambda a.$$

Thus,

$$2Ca - 2\lambda a = 0,$$

so

$$Ca = \lambda a.$$

Theorem 6.2.1 (First Principal Component)**.** *The direction of maximum variance is the eigenvector a_1 corresponding to the largest eigenvalue λ_1 of C:*

$$Ca_1 = \lambda_1 a_1.$$

The maximum variance is

$$a_1^T C a_1 = \lambda_1.$$

Proof. From the Lagrange multiplier derivation, any stationary direction satisfies

$$Ca = \lambda a.$$

If a is normalized so that $a^T a = 1$, then

$$a^T C a = a^T \lambda a = \lambda a^T a = \lambda.$$

Therefore, maximizing $a^T C a$ over unit vectors is equivalent to choosing the largest eigenvalue. The associated eigenvector is the first principal component. □

6.3 Second Principal Component

The second principal component is the direction of maximum variance subject to being uncorrelated with the first principal component.

Let a_1 be the first principal component. Since

$$\mathrm{Cov}(a^T X, a_1^T X) = a^T C a_1,$$

the uncorrelatedness constraint is

$$a^T C a_1 = 0.$$

The constrained optimization problem is

$$\max_{a \in \mathbb{R}^N} \left[a^T C a - \lambda (a^T a - 1) - \lambda' (a^T C a_1) \right].$$

The first-order condition is

$$2Ca - 2\lambda a - \lambda' C a_1 = 0.$$

Since

$$Ca_1 = \lambda_1 a_1,$$

left multiply by a_1^T:

$$2a_1^T C a - 2\lambda a_1^T a - \lambda' a_1^T C a_1 = 0.$$

The orthogonality and uncorrelatedness constraints imply

$$a_1^T C a = 0, \quad a_1^T a = 0.$$

Therefore,

$$-\lambda' a_1^T C a_1 = 0.$$

But

$$a_1^T C a_1 = \lambda_1,$$

so

$$\lambda' = 0.$$

Thus the first-order condition reduces to

$$Ca = \lambda a.$$

Theorem 6.3.1 (Second Principal Component)**.** *The second principal component is the eigenvector a_2 corresponding to the second largest eigenvalue λ_2:*

$$Ca_2 = \lambda_2 a_2.$$

6.4 Principal Components Analysis

Let

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0$$

be the eigenvalues of C, with corresponding orthonormal eigenvectors

$$e_1, \ldots, e_N.$$

Since C is symmetric positive semidefinite, it admits the spectral decomposition

$$C = \sum_{k=1}^{N} \lambda_k e_k e_k^T.$$

For any eigenvector e_l,

$$Ce_l = \sum_{k=1}^{N} \lambda_k e_k (e_k^T e_l).$$

Since the eigenvectors are orthonormal,

$$e_k^T e_l = \delta_{kl}.$$

Therefore,

$$Ce_l = \lambda_l e_l.$$

In matrix form, let

$$E = \begin{pmatrix} e_1 & e_2 & \cdots & e_N \end{pmatrix},$$

and

$$\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_N).$$

Then

$$C = E\Lambda E^T.$$

Definition 6.4.1 (Principal Components)**.** The principal components are the orthonormal eigenvectors of the covariance matrix, ordered by decreasing eigenvalue.

Remark 6.4.1. The eigenvectors associated with the largest eigenvalues capture the directions of greatest variance in the data.

6.5 Low-Rank Approximation

If only $K < N$ principal components are retained, then

$$C \approx C_K = \sum_{k=1}^{K} \lambda_k e_k e_k^T.$$

Equivalently,

$$C_K = E_K \Lambda_K E_K^T,$$

where

$$E_K = \begin{pmatrix} e_1 & \cdots & e_K \end{pmatrix}$$

and

$$\Lambda_K = \operatorname{diag}(\lambda_1, \ldots, \lambda_K).$$

The remaining covariance is

$$C - C_K = \sum_{l=K+1}^{N} \lambda_l e_l e_l^T.$$

Definition 6.5.1 (Projection Matrix)**.** The rank-K PCA projection matrix is

$$P_K = E_K E_K^T.$$

Remark 6.5.1. The matrix P_K projects observations onto the subspace spanned by the first K principal components.

6.6 New Coordinate System

Since E is orthonormal,

$$E^T E = E E^T = I.$$

Thus, the transformation

$$Z = E^T X$$

expresses X in the principal component coordinate system.

Using only the first K eigenvectors,

$$Z_K = E_K^T X.$$

The reconstruction in the original coordinates is

$$X_K = E_K Z_K.$$

Therefore,

$$X_K = E_K E_K^T X = P_K X.$$

Remark 6.6.1. PCA can be viewed as a rotation followed by truncation. The full transformation $E^T X$ preserves all information, while the truncated transformation $E_K^T X$ discards directions with smaller variance.

6.7 Samples and the Sample Covariance Matrix

Suppose we have n centered observations

$$x_1, \dots, x_n \in \mathbb{R}^N,$$

arranged as columns of a data matrix

$$X = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}.$$

Assuming the sample mean is zero, the sample covariance matrix is

$$C = \frac{1}{n-1} \sum_{i=1}^{n} x_i x_i^T.$$

In matrix form,

$$C = \frac{1}{n-1} X X^T.$$

6.8 Connection to Singular Value Decomposition

Let the singular value decomposition of X be

$$X = UWV^T,$$

where

$$U^T U = I, \quad V^T V = I,$$

and W is diagonal with nonnegative singular values.

Then

$$C = \frac{1}{n-1} X X^T.$$

Substituting the SVD,

$$C = \frac{1}{n-1} UWV^T VWU^T.$$

Since

$$V^T V = I,$$

we obtain

$$C = \frac{1}{n-1} UW^2 U^T.$$

Thus, if

$$C = E \Lambda E^T,$$

then

$$E = U,$$

and

$$\Lambda = \frac{1}{n-1} W^2.$$

Remark 6.8.1. Computing PCA by SVD is often more numerically stable than explicitly forming the covariance matrix.

6.9 Whitening

Definition 6.9.1 (Whitening Transformation)**.** Given covariance decomposition

$$C = E\Lambda E^T,$$

the whitened representation of centered data X is

$$A = \Lambda^{-1/2} E^T X.$$

Proposition 6.9.1. *If X has covariance $C = E\Lambda E^T$, then the whitened variable*

$$A = \Lambda^{-1/2} E^T X$$

has identity covariance.

Proof. Compute

$$\mathrm{Cov}(A) = \mathbb{E}[AA^T].$$

Substituting $A = \Lambda^{-1/2} E^T X$,

$$\mathrm{Cov}(A) = \mathbb{E}[\Lambda^{-1/2} E^T X X^T E \Lambda^{-1/2}].$$

Since $\mathbb{E}[XX^T] = C$,

$$\mathrm{Cov}(A) = \Lambda^{-1/2} E^T C E \Lambda^{-1/2}.$$

Using $C = E\Lambda E^T$,

$$\mathrm{Cov}(A) = \Lambda^{-1/2} E^T E \Lambda E^T E \Lambda^{-1/2}.$$

Since $E^T E = I$,

$$\mathrm{Cov}(A) = \Lambda^{-1/2} \Lambda \Lambda^{-1/2} = I.$$

□

6.10 Scree Plot and Explained Variance

The eigenvalue spectrum is

$$\{\lambda_1, \lambda_2, \ldots, \lambda_N\}.$$

A scree plot displays these eigenvalues in decreasing order.

Definition 6.10.1 (Explained Variance Ratio)**.** The explained variance ratio of component k is

$$\frac{\lambda_k}{\sum_{j=1}^{N} \lambda_j}.$$

The cumulative explained variance of the first K components is

$$\frac{\sum_{k=1}^{K} \lambda_k}{\sum_{j=1}^{N} \lambda_j}.$$

Remark 6.10.1. A common heuristic is to choose K by looking for an elbow in the scree plot. The elbow indicates that additional components contribute relatively little variance.

Proposition 6.10.1 (Variance Additivity for Uncorrelated Components)**.** *If $Z_1, \ldots, Z_K$ are pairwise uncorrelated, then*

$$\mathrm{Var}\left(\sum_{k=1}^{K} Z_k\right) = \sum_{k=1}^{K} \mathrm{Var}(Z_k).$$

Proof. For two variables,

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\mathrm{Cov}(X,Y).$$

If X and Y are uncorrelated, then $\mathrm{Cov}(X,Y) = 0$, so

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y).$$

The general result follows by applying the same expansion to K variables:

$$\mathrm{Var}\left(\sum_{k=1}^{K} Z_k\right) = \sum_{k=1}^{K} \mathrm{Var}(Z_k) + 2\sum_{i<j} \mathrm{Cov}(Z_i, Z_j).$$

If all variables are pairwise uncorrelated, all covariance terms vanish. □

6.11 PCA Algorithm

Algorithm 3 Principal Components Analysis

Require: Data matrix $X \in \mathbb{R}^{n\times d}$ with rows as observations, number of components K
Ensure: Principal component scores Z_K, loading matrix E_K
1: Center the data: $X_c \leftarrow X - \bar{X}$
2: Compute covariance matrix: $C \leftarrow \frac{1}{n-1} X_c^T X_c$
3: Compute eigendecomposition: $C = E\Lambda E^T$
4: Sort eigenvalues in decreasing order and reorder eigenvectors accordingly
5: Keep first K eigenvectors: $E_K \leftarrow [e_1, \ldots, e_K]$
6: Project data: $Z_K \leftarrow X_c E_K$
7: **return** Z_K, E_K

Algorithm 4 PCA via Singular Value Decomposition

Require: Centered data matrix $X_c \in \mathbb{R}^{n\times d}$, number of components K
Ensure: Principal component scores Z_K, loading matrix V_K
1: Compute SVD: $X_c = UWV^T$
2: Keep first K right singular vectors: $V_K \leftarrow [v_1, \ldots, v_K]$
3: Compute scores: $Z_K \leftarrow X_c V_K$
4: Eigenvalues are $\lambda_k = \frac{w_k^2}{n-1}$
5: **return** Z_K, V_K

6.12 Implementation Notes

```python
import numpy as np

def pca(X, K):
    """
    PCA using SVD.
    Rows are observations and columns are features.
    """
    X_centered = X - X.mean(axis=0)

    U, S, Vt = np.linalg.svd(X_centered, full_matrices=False)

    components = Vt[:K]
    scores = X_centered @ components.T
    eigenvalues = (S**2) / (X.shape[0] - 1)

    explained_variance_ratio = eigenvalues / eigenvalues.sum()

    return scores, components, eigenvalues[:K],
       explained_variance_ratio[:K]
```

Remark 6.12.1. For machine learning data matrices, the common convention is rows as observations and columns as features. In this convention, the covariance matrix is

$$C = \frac{1}{n-1} X_c^T X_c.$$

In the column-vector convention used in the derivation above, the covariance matrix is

$$C = \frac{1}{n-1} X_c X_c^T.$$

The mathematics is the same, but the orientation of the data matrix changes.

Chapter 7

Nearest Neighbors

7.1 Classification

Classification is the supervised learning problem of assigning discrete labels to observations.

A training set consists of labeled data:

$$T = \{(x_i, C_i)\}_{i=1}^{N},$$

where

$$x_i \in \mathbb{R}^d$$

is a feature vector and C_i is the known class label.

A query set consists of unlabeled data:

$$Q = \{x_i\}_{i=1}^{M},$$

where

$$x_i \in \mathbb{R}^d.$$

The goal is to construct a classifier

$$\hat{C}(x)$$

that assigns a class label to a new point x.

Remark 7.1.1. Classification is similar to regression, but the response variable is discrete rather than continuous. Common classifiers include nearest neighbors, naive Bayes, LDA/QDA, logistic regression, decision trees, random forests, and support vector machines.

7.2 Nearest Neighbors

Definition 7.2.1 (Nearest Neighbor Classifier)**.** Given a query point x, the nearest neighbor classifier assigns x the label of the closest training point:

$$\hat{C}(x) = C_{i^\star},$$

where

$$i^\star = \arg\min_{1 \le i \le N} d(x, x_i).$$

The most common distance is Euclidean distance:

$$d(x, x_i) = \|x - x_i\|_2 = \sqrt{\sum_{j=1}^{d} (x_j - x_{ij})^2}.$$

7.3 k-Nearest Neighbors

Definition 7.3.1 (k-Nearest Neighbor Classifier)**.** Let $\mathcal{N}_k(x)$ denote the indices of the k nearest training points to x. The k-nearest neighbor classifier assigns the most frequent class among those neighbors:

$$\hat{C}(x) = \arg\max_c \sum_{i \in \mathcal{N}_k(x)} \mathbb{1}(C_i = c).$$

A weighted version uses distance-dependent weights:

$$\hat{C}(x) = \arg\max_c \sum_{i \in \mathcal{N}_k(x)} w_i(x) \mathbb{1}(C_i = c),$$

where a common choice is

$$w_i(x) = \frac{1}{d(x, x_i) + \varepsilon}.$$

Remark 7.3.1. Using k instead of a fixed distance cutoff adapts better to regions with different data density. In dense regions, the neighborhood is small; in sparse regions, the neighborhood expands.

7.4 Bias-Variance Tradeoff

The choice of k controls model complexity.

- Small k: low bias, high variance
- Large k: high bias, low variance

Remark 7.4.1. The case $k = 1$ produces highly flexible decision boundaries, but it is sensitive to noise. Larger values of k smooth the decision boundary.

7.5 Evaluating on a Grid

To visualize a classifier in two dimensions:

1. Create a mesh of query points with resolution h.

2. Apply the classifier to each grid point.
3. Plot the predicted class regions.
4. Overlay the training data.

This produces a visualization of the decision boundary.

Algorithm 5 k-Nearest Neighbors Classification

Require: Training data $\{(x_i, C_i)\}_{i=1}^{N}$, query point x, number of neighbors k
Ensure: Predicted class label $\hat{C}(x)$
1: Compute distances $d_i \leftarrow d(x, x_i)$ for $i = 1, \ldots, N$
2: Sort distances in increasing order
3: Let $\mathcal{N}_k(x)$ be the indices of the first k points
4: Count class votes among $\{C_i : i \in \mathcal{N}_k(x)\}$
5: Return the class with the largest vote count

7.6 Meaningful Distance

Nearest neighbor methods depend entirely on the distance function. If features have different units or scales, Euclidean distance can be dominated by variables with larger numerical ranges.

For example, suppose

$$x = (\text{height in meters}, \text{income in dollars}).$$

The income coordinate may dominate the distance unless features are scaled.

Definition 7.6.1 (Standardization)**.** Given feature j, standardization transforms

$$x_{ij} \mapsto \frac{x_{ij} - \bar{x}_j}{s_j},$$

where $\bar{x}_j$ is the sample mean of feature j and s_j is its sample standard deviation.

Remark 7.6.1. Centering and scaling are often essential preprocessing steps for nearest neighbor methods.

7.7 Curse of Dimensionality

In high dimensions, nearest neighbor methods become less effective because distances become less informative.

Remark 7.7.1. Informally, in high dimensions, "everybody is lonely." Points tend to be far apart, and the contrast between nearest and farthest neighbors decreases.

7.7.1 Volume Concentration

Consider a unit ball in $\mathbb{R}^d$. The volume concentrates near the boundary as d increases. For a point uniformly distributed in the unit ball, the probability that its radius is at most r is

$$\mathbb{P}(R \leq r) = r^d.$$

Thus,

$$\mathbb{P}(R > r) = 1 - r^d.$$

For fixed $r < 1$,

$$r^d \to 0 \quad \text{as} \quad d \to \infty.$$

Therefore, most of the volume lies near the surface.

Remark 7.7.2. This explains why nearest neighbor methods may require very large sample sizes in high dimensions. The data become sparse relative to the feature space.

7.8 Implementation Notes

```python
import numpy as np

def knn_predict(X_train, y_train, x_query, k=5):
    """
    Simple k-nearest neighbors classifier.
    X_train: array of shape (N, d)
    y_train: array of shape (N,)
    x_query: array of shape (d,)
    """
    distances = np.linalg.norm(X_train - x_query, axis=1)
    neighbor_idx = np.argsort(distances)[:k]
    neighbor_labels = y_train[neighbor_idx]

    labels, counts = np.unique(neighbor_labels, return_counts=True)
    return labels[np.argmax(counts)]
```

```python
def standardize_train_test(X_train, X_test):
    """
    Standardize using training-set statistics only.
    """
    mean = X_train.mean(axis=0)
    std = X_train.std(axis=0, ddof=1)

    X_train_scaled = (X_train - mean) / std
    X_test_scaled = (X_test - mean) / std

    return X_train_scaled, X_test_scaled
```

Chapter 8

Naive Bayes, LDA, and QDA

8.1 Naive Bayes Classifier

8.1.1 Bayesian Classification

Let $\{C_k\}_{k=1}^K$ denote discrete classes and $x \in \mathbb{R}^d$ a feature vector.

By Bayes' rule,

$$\mathbb{P}(C_k \mid x) = \frac{\pi_k \, \mathcal{L}_x(C_k)}{Z},$$

where

$$\pi_k = \mathbb{P}(C_k) \quad \text{(prior)}, \quad \mathcal{L}_x(C_k) = p(x \mid C_k) \quad \text{(likelihood)},$$

and Z is a normalization constant independent of k.

Proposition 8.1.1 (MAP Classification Rule)**.** *The Bayes classifier assigns*

$$\hat{k} = \arg\max_k \mathbb{P}(C_k \mid x) = \arg\max_k \left[\pi_k \, p(x \mid C_k)\right].$$

Remark 8.1.1. The normalization constant cancels in the arg max, so it need not be computed explicitly.

8.1.2 Naive Independence Assumption

Definition 8.1.1 (Naive Bayes Model)**.** Naive Bayes assumes conditional independence of features:

$$p(x \mid C_k) = \prod_{\alpha=1}^{d} p(x_\alpha \mid C_k).$$

Thus, the classifier becomes

$$\hat{k} = \arg\max_k \left[\pi_k \prod_{\alpha=1}^{d} p(x_\alpha \mid C_k)\right].$$

8.1.3 Gaussian Naive Bayes

Assume each feature is conditionally Gaussian:

$$p(x_\alpha \mid C_k) = \frac{1}{\sqrt{2\pi\sigma_{k,\alpha}^2}} \exp\left(-\frac{(x_\alpha - \mu_{k,\alpha})^2}{2\sigma_{k,\alpha}^2}\right).$$

Definition 8.1.2 (Parameter Estimation)**.** For each class k and feature α, estimate

$$\mu_{k,\alpha} = \frac{1}{N_k} \sum_{i:C_i=k} x_{i\alpha}, \quad \sigma_{k,\alpha}^2 = \frac{1}{N_k - 1} \sum_{i:C_i=k} (x_{i\alpha} - \mu_{k,\alpha})^2.$$

Class priors are estimated as

$$\pi_k = \frac{N_k}{N}.$$

8.1.4 Log-Domain Form

To improve numerical stability, take logarithms:

$$\hat{k} = \arg\max_k \left[\log \pi_k + \sum_{\alpha=1}^{d} \log p(x_\alpha \mid C_k)\right].$$

Remark 8.1.2. Working in the log-domain avoids numerical underflow from multiplying many small probabilities.

8.1.5 Advantages and Limitations

Proposition 8.1.2 (Scaling Invariance)**.** *Naive Bayes is invariant to feature scaling (for Gaussian likelihoods), since each feature is normalized by its variance.*

Remark 8.1.3. The independence assumption is often unrealistic. However, naive Bayes performs well in practice due to low variance and computational efficiency.

8.2 Discriminant Analysis

Discriminant analysis assumes class-conditional Gaussian distributions:

$$p(x \mid C_k) = \mathcal{N}(x; \mu_k, \Sigma_k).$$

8.2.1 Quadratic Discriminant Analysis (QDA)

Definition 8.2.1 (QDA Model)**.** Each class has its own covariance matrix:

$$p(x \mid C_k) = \frac{1}{\sqrt{(2\pi)^d |\Sigma_k|}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)\right).$$

The discriminant function is obtained from the log-posterior:

$$\delta_k(x) = \log \pi_k - \frac{1}{2} \log |\Sigma_k| - \frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k).$$

Proposition 8.2.1 (QDA Classification Rule)**.**

$$\hat{k} = \arg\max_k \delta_k(x).$$

Remark 8.2.1. The decision boundary between two classes is quadratic in x due to the quadratic form $(x - \mu_k)^T \Sigma_k^{-1}(x - \mu_k)$.

8.2.2 Binary QDA Decision Boundary

For two classes C_1 and C_2, compare

$$(x - \mu_1)^T \Sigma_1^{-1}(x - \mu_1) + \log |\Sigma_1|$$

and

$$(x - \mu_2)^T \Sigma_2^{-1}(x - \mu_2) + \log |\Sigma_2|.$$

The classification depends on which quantity is smaller (after including priors).

8.3 Linear Discriminant Analysis (LDA)

Definition 8.3.1 (LDA Model)**.** LDA assumes a common covariance matrix across classes:

$$\Sigma_k = \Sigma \quad \forall k.$$

The discriminant function becomes

$$\delta_k(x) = \log \pi_k - \frac{1}{2}(x - \mu_k)^T \Sigma^{-1}(x - \mu_k).$$

8.3.1 Linearization of the Decision Boundary

Expand the quadratic term:

$$(x - \mu_k)^T \Sigma^{-1}(x - \mu_k) = x^T \Sigma^{-1} x - 2\mu_k^T \Sigma^{-1} x + \mu_k^T \Sigma^{-1} \mu_k.$$

When comparing two classes, the $x^T \Sigma^{-1} x$ term cancels, yielding a linear function of x:

$$\delta_k(x) = \mu_k^T \Sigma^{-1} x - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k.$$

Theorem 8.3.1 (LDA Decision Boundary)**.** *The decision boundary between classes is linear in x.*

Proof. The discriminant difference between two classes is

$$\delta_1(x) - \delta_2(x) = (\mu_1 - \mu_2)^T \Sigma^{-1} x - \frac{1}{2}\left(\mu_1^T \Sigma^{-1} \mu_1 - \mu_2^T \Sigma^{-1} \mu_2\right) + \log \frac{\pi_1}{\pi_2}.$$

This is an affine (linear + constant) function of x, so the boundary

$$\delta_1(x) = \delta_2(x)$$

is linear. □

8.3.2 Parameter Estimation

Means:

$$\mu_k = \frac{1}{N_k} \sum_{i:C_i=k} x_i.$$

Shared covariance:

$$\Sigma = \frac{1}{N-K} \sum_{k=1}^{K} \sum_{i:C_i=k} (x_i - \mu_k)(x_i - \mu_k)^T.$$

8.3.3 Comparison: LDA vs QDA

- QDA:
  - Separate covariance matrices Σ_k
  - Flexible (quadratic boundaries)
  - High variance (many parameters)
- LDA:
  - Shared covariance Σ
  - Linear boundaries
  - Lower variance, better for small sample sizes

Remark 8.3.1. The choice between LDA and QDA depends on whether class covariances are similar. If covariances differ significantly, QDA may perform better; otherwise LDA is more stable.

8.4 Algorithmic Summary

Algorithm 6 Gaussian Naive Bayes

Require: Training data $\{(x_i, C_i)\}$, query point x
Ensure: Predicted class $\hat{k}$
1: Estimate $\mu_{k,\alpha}$ and $\sigma^2_{k,\alpha}$ for each class and feature
2: Estimate priors π_k
3: **for** each class k **do**
4: Compute score:

$$s_k = \log \pi_k + \sum_{\alpha} \log p(x_\alpha \mid C_k)$$

5: **end for**
6: Return $\arg\max_k s_k$

Algorithm 7 LDA Classification

Require: Training data $\{(x_i, C_i)\}$, query point x
Ensure: Predicted class $\hat{k}$
1: Estimate μ_k, shared Σ, and priors π_k
2: **for** each class k **do**
3: Compute

$$\delta_k(x) = \mu_k^T \Sigma^{-1} x - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log \pi_k$$

4: **end for**
5: Return $\arg\max_k \delta_k(x)$

Algorithm 8 QDA Classification

Require: Training data $\{(x_i, C_i)\}$, query point x
Ensure: Predicted class $\hat{k}$
1: Estimate μ_k, Σ_k, and priors π_k
2: **for** each class k **do**
3: Compute

$$\delta_k(x) = \log \pi_k - \frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k)$$

4: **end for**
5: Return $\arg\max_k \delta_k(x)$

8.5 Implementation Notes

```
1 import numpy as np
2
```

```python
def gaussian_naive_bayes_predict(X_train, y_train, x):
    classes = np.unique(y_train)
    scores = []

    for k in classes:
        X_k = X_train[y_train == k]
        mu = X_k.mean(axis=0)
        var = X_k.var(axis=0, ddof=1)

        log_prior = np.log(len(X_k) / len(X_train))
        log_likelihood = -0.5 * np.sum(
            np.log(2*np.pi*var) + ((x - mu)**2)/var
        )

        scores.append(log_prior + log_likelihood)

    return classes[np.argmax(scores)]
```

```python
def lda_predict(X_train, y_train, x):
    classes = np.unique(y_train)
    n, d = X_train.shape

    means = {}
    priors = {}

    for k in classes:
        X_k = X_train[y_train == k]
        means[k] = X_k.mean(axis=0)
        priors[k] = len(X_k) / n

    # shared covariance
    X_centered = []
    for k in classes:
        X_k = X_train[y_train == k]
        X_centered.append(X_k - means[k])
    X_centered = np.vstack(X_centered)

    Sigma = np.cov(X_centered, rowvar=False)
    Sigma_inv = np.linalg.inv(Sigma)

    scores = []
    for k in classes:
        mu = means[k]
        score = (
            mu @ Sigma_inv @ x
            - 0.5 * mu @ Sigma_inv @ mu
            + np.log(priors[k])
        )
```

```
31            scores.append(score)
32
33        return classes[np.argmax(scores)]
```

Chapter 9

Cross Validation

9.1 Model Evaluation

Given a learning algorithm that produces an estimator $\hat{f}$ from training data, we seek to estimate its predictive performance on unseen data.

Definition 9.1.1 (Generalization Error)**.** Let (X, Y) be a new observation drawn from the same distribution as the training data. The generalization error is

$$\mathcal{E} = \mathbb{E}\left[L(Y, \hat{f}(X))\right],$$

where $L(\cdot, \cdot)$ is a loss function (e.g., squared error for regression or 0-1 loss for classification).

Since the true distribution is unknown, we approximate $\mathcal{E}$ using resampling methods.

9.2 Train-Test Split and Complementary Subsets

Definition 9.2.1 (Hold-Out Method)**.** Split the data into two disjoint subsets:

- Training set $\mathcal{D}_{\text{train}}$
- Test set $\mathcal{D}_{\text{test}}$

Train the model on $\mathcal{D}_{\text{train}}$ and evaluate on $\mathcal{D}_{\text{test}}$.

Remark 9.2.1. The test error is

$$\widehat{\mathcal{E}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(x_i, y_i) \in \mathcal{D}_{\text{test}}} L(y_i, \hat{f}(x_i)).$$

Remark 9.2.2. A single split can have high variance depending on how the data is partitioned. To reduce this variance, one may use repeated random splits (complementary subsets).

9.3 Repeated Random Subsampling

Definition 9.3.1 (Repeated Hold-Out)**.** Repeat the following procedure multiple times:

1. Randomly split data into training and test sets.
2. Train the model on the training set.
3. Evaluate on the test set.

Average the resulting test errors.

Remark 9.3.1. This reduces variance relative to a single split but may reuse data unevenly across training and testing.

9.4 Leave-One-Out Cross Validation (LOOCV)

Definition 9.4.1 (LOOCV)**.** For a dataset of size n, perform n training rounds. In each round:

- Use $n-1$ observations for training
- Use the remaining observation for testing

The LOOCV estimate of prediction error is

$$\widehat{\mathcal{E}}_{\text{LOOCV}} = \frac{1}{n}\sum_{i=1}^{n} L\Big(y_i, \hat{f}^{(-i)}(x_i)\Big),$$

where $\hat{f}^{(-i)}$ is the model trained without the i-th observation.

Remark 9.4.1. LOOCV has low bias because nearly the full dataset is used for training, but it can be computationally expensive.

Remark 9.4.2. For some models (e.g., linear regression), LOOCV can be computed efficiently without retraining using analytic formulas involving the hat matrix.

9.5 k-Fold Cross Validation

Definition 9.5.1 (k-Fold Cross Validation)**.** Partition the dataset into k disjoint subsets (folds) of approximately equal size:

$$\mathcal{D} = \mathcal{D}_1 \cup \cdots \cup \mathcal{D}_k.$$

For each fold $j = 1, \ldots, k$:

- Train on $\mathcal{D} \setminus \mathcal{D}_j$
- Test on $\mathcal{D}_j$

The cross-validation estimate is

$$\widehat{\mathcal{E}}_{\text{CV}} = \frac{1}{k}\sum_{j=1}^{k}\frac{1}{|\mathcal{D}_j|}\sum_{(x_i,y_i)\in\mathcal{D}_j} L\Big(y_i, \hat{f}^{(-j)}(x_i)\Big).$$

Remark 9.5.1. Common choices are $k = 5$ or $k = 10$. Smaller k increases bias but reduces computational cost; larger k reduces bias but increases variance and computation.

Proposition 9.5.1. *LOOCV is a special case of k-fold cross validation with $k = n$.*

9.6 Bias-Variance Tradeoff in Cross Validation

- LOOCV:
  - Low bias
  - High variance
  - High computational cost
- Small k (e.g., $k = 2$):
  - Higher bias
  - Lower variance
  - Lower computational cost

9.7 Cross Validation Algorithm

Algorithm 9 k-Fold Cross Validation

Require: Dataset $\mathcal{D}$, number of folds k
Ensure: Estimated prediction error $\widehat{\mathcal{E}}$
1: Partition $\mathcal{D}$ into k folds $\{\mathcal{D}_j\}_{j=1}^{k}$
2: **for** $j = 1$ to k **do**
3: Train model $\hat{f}^{(-j)}$ on $\mathcal{D} \setminus \mathcal{D}_j$
4: Compute error on $\mathcal{D}_j$
5: **end for**
6: Average the errors across folds
7: **return** $\widehat{\mathcal{E}}$

9.8 Implementation Notes

```python
from sklearn.model_selection import KFold
import numpy as np

def k_fold_cv(model, X, y, k=5):
    kf = KFold(n_splits=k, shuffle=True, random_state=0)
    errors = []

    for train_idx, test_idx in kf.split(X):
        X_train, X_test = X[train_idx], X[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

        error = np.mean((y_test - y_pred)**2)
        errors.append(error)

    return np.mean(errors)
```

Remark 9.8.1. In practice, cross validation is used not only to estimate model performance but also to tune hyperparameters (e.g., k in k-NN or λ in regularization).

Chapter 10

Decision Trees

10.1 Tree Structures

Definition 10.1.1 (Tree)**.** A tree is a connected acyclic graph with a hierarchical structure.

Definition 10.1.2 (Binary Tree)**.** A binary tree is a tree in which each node has at most two children:

- A single **root** node
- Each node has at most a **left** and **right** child
- **Leaves** are terminal nodes with no children

Remark 10.1.1. Trees are naturally defined recursively: each subtree is itself a tree.

10.1.1 n-ary Trees

Definition 10.1.3 (n-ary Tree)**.** An n-ary tree allows each node to have up to n children.

10.2 Trees in Computation

Trees appear in many applications:

- Search structures: kd-trees, B-trees, R-trees, ball trees
- Decision making: classification and regression trees

10.2.1 kd-Trees

Definition 10.2.1 (kd-Tree)**.** A kd-tree is a binary tree that recursively partitions $\mathbb{R}^d$ by splitting along coordinate axes.

Remark 10.2.1. At each level, the split dimension cycles through coordinates, producing a space-partitioning structure.

10.3 Decision Trees

10.3.1 Recursive Partitioning

A decision tree partitions the feature space recursively.

At a node with dataset D, we choose a split

$$\theta = (j, t),$$

where j is a feature index and t is a threshold.

The split defines two subsets:

$$D_{\text{left}}(\theta) = \{x \in D : x_j \leq t\}, \quad D_{\text{right}}(\theta) = \{x \in D : x_j > t\}.$$

10.3.2 Impurity Minimization

We choose θ to minimize impurity:

$$I(\theta) = \frac{n_{\text{left}}}{n} H(D_{\text{left}}(\theta)) + \frac{n_{\text{right}}}{n} H(D_{\text{right}}(\theta)),$$

where $H(\cdot)$ measures impurity and $n = |D|$.

Definition 10.3.1 (Impurity Function)**.** An impurity function measures how mixed the class labels are in a dataset.

10.3.3 Gini Impurity

Definition 10.3.2 (Gini Impurity)**.** For a dataset D with class proportions $p_1, \ldots, p_K$,

$$H(D) = \sum_{k=1}^{K} p_k(1 - p_k).$$

Remark 10.3.1. Equivalently,

$$H(D) = 1 - \sum_{k=1}^{K} p_k^2.$$

10.3.4 Example: Root Impurity

Suppose a dataset contains 8 observations across 3 classes with counts $(4, 3, 1)$. Then

$$H = \frac{4}{8}\left(1 - \frac{4}{8}\right) + \frac{3}{8}\left(1 - \frac{3}{8}\right) + \frac{1}{8}\left(1 - \frac{1}{8}\right).$$

Compute each term:

$$\frac{4}{8}\cdot\frac{4}{8}=\frac{16}{64},\quad \frac{3}{8}\cdot\frac{5}{8}=\frac{15}{64},\quad \frac{1}{8}\cdot\frac{7}{8}=\frac{7}{64}.$$

Thus,

$$H=\frac{16}{64}+\frac{15}{64}+\frac{7}{64}=\frac{38}{64}=\frac{19}{32}\approx 0.59375.$$

10.3.5 Example: Split Impurity

Suppose a split produces two partitions:

Left partition: 2 samples, all from one class:

$$H_{\text{left}}=\frac{2}{2}(1-\frac{2}{2})=0.$$

Right partition: 6 samples with counts $(3,2,1)$:

$$H_{\text{right}}=\frac{3}{6}\left(1-\frac{3}{6}\right)+\frac{2}{6}\left(1-\frac{2}{6}\right)+\frac{1}{6}\left(1-\frac{1}{6}\right).$$

Compute:

$$=\frac{3}{6}\cdot\frac{3}{6}+\frac{2}{6}\cdot\frac{4}{6}+\frac{1}{6}\cdot\frac{5}{6}=\frac{9}{36}+\frac{8}{36}+\frac{5}{36}=\frac{22}{36}=\frac{11}{18}\approx 0.6111.$$

10.3.6 Weighted Impurity

The total impurity after the split is

$$I(\theta)=\frac{2}{8}\cdot 0+\frac{6}{8}\cdot\frac{11}{18}=\frac{6}{8}\cdot\frac{11}{18}=\frac{66}{144}=\frac{11}{24}.$$

Remark 10.3.2. Impurity must be weighted by partition size. A perfectly pure small partition does not necessarily yield a good split overall.

10.4 Regression Trees

For regression, impurity is measured by variance:

$$H(D)=\frac{1}{|D|}\sum_{x_i\in D}(y_i-\bar{y})^2.$$

10.5 Greedy Tree Construction

Decision trees are built greedily:

- At each node, choose the best split minimizing impurity

- Recurse on resulting subsets
- Stop when a stopping criterion is met

Algorithm 10 Decision Tree Training

Require: Dataset D, stopping criterion
Ensure: Decision tree
if stopping criterion satisfied **then**
    Return leaf node with prediction
end if
for each feature j and threshold t **do**
    Compute split $\theta = (j, t)$
    Compute impurity $I(\theta)$
end for
Choose $\theta^\star = \arg\min_\theta I(\theta)$
Split data into $D_{\text{left}}, D_{\text{right}}$
Recursively build left and right subtrees
Return node with split $\theta^\star$

10.6 Properties of Decision Trees

- Nonparametric: no assumption on data distribution
- Capture nonlinear relationships
- Easy to interpret
- Can overfit without regularization

10.7 Regularization and Practical Considerations

Common techniques:

- Maximum tree depth
- Minimum samples per leaf
- Pruning (post-training simplification)

Remark 10.7.1. Decision trees have low bias but high variance. Ensemble methods such as random forests reduce variance.

10.8 Implementation Notes

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(max_depth=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Chapter 11

Scikit-Learn Pipeline and Random Forests

11.1 Scikit-Learn Pipeline and Grid Search

11.1.1 Pipelines

Definition 11.1.1 (Pipeline)**.** A pipeline chains multiple transformations and a final estimator into a single object:

$$X \to \text{Transform}_1 \to \cdots \to \text{Transform}_m \to \hat{f}.$$

Remark 11.1.1. Pipelines ensure that preprocessing steps (e.g., scaling, PCA) are applied consistently during training and testing.

Example 11.1.1 (Pipeline Structure)**.**

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=5)),
    ("model", LogisticRegression())
])
```

11.1.2 Grid Search with Cross Validation

Definition 11.1.2 (Grid Search)**.** Grid search selects hyperparameters by evaluating a model over a predefined parameter grid using cross validation.

Example 11.1.2 (Grid Search with Pipeline)**.**

```
from sklearn.model_selection import GridSearchCV
```

```
2
3  param_grid = {
4      "pca__n_components": [2, 5, 10],
5      "model__C": [0.1, 1, 10]
6  }
7
8  grid = GridSearchCV(pipe, param_grid, cv=5)
9  grid.fit(X_train, y_train)
10
11 best_model = grid.best_estimator_
```

Remark 11.1.2. The notation `step__param` specifies a parameter for a given pipeline step.

Remark 11.1.3. Grid search combined with cross validation provides a principled way to tune hyperparameters while avoiding overfitting to a single train-test split.

11.2 Random Forests

11.2.1 Random Trees

Definition 11.2.1 (Randomized Tree)**.** A randomized decision tree selects a random subset of features when searching for the best split at each node.

Remark 11.2.1. In high dimensions, searching all features is computationally expensive. Random feature selection reduces cost and introduces diversity across trees.

11.2.2 Forest of Random Trees

Definition 11.2.2 (Random Forest)**.** A random forest is an ensemble of randomized decision trees:

$$\{\hat{f}_1, \ldots, \hat{f}_B\}.$$

Predictions are combined:

- Classification: majority vote
- Regression: average prediction

Definition 11.2.3 (Bootstrap Sampling)**.** Each tree is trained on a bootstrap sample of the data, obtained by sampling with replacement.

11.2.3 Bagging and Variance Reduction

Proposition 11.2.1 (Variance Reduction via Averaging)**.** *Let $\hat{f}_1, \ldots, \hat{f}_B$ be independent estimators with variance σ^2. Then the average*

$$\bar{f} = \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b$$

has variance

$$\mathrm{Var}(\bar{f}) = \frac{\sigma^2}{B}.$$

Proof. Using independence,

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B}\hat{f}_b\right) = \frac{1}{B^2}\sum_{b=1}^{B}\mathrm{Var}(\hat{f}_b) = \frac{B\sigma^2}{B^2} = \frac{\sigma^2}{B}.$$

□

Remark 11.2.2. In practice, trees are not independent, but randomization reduces correlation, improving variance reduction.

11.2.4 Connection to the Central Limit Theorem

Remark 11.2.3. As the number of trees B increases, the average prediction stabilizes. Under mild conditions, the distribution of the ensemble prediction approaches a normal distribution due to averaging effects.

11.3 Feature Selection in Random Forests

Definition 11.3.1 (Feature Importance)**.** Feature importance measures how often and how effectively a feature is used to split the data.

A common measure is the total impurity reduction contributed by a feature across all trees.

Remark 11.3.1. Features that frequently produce large impurity reductions are considered more important.

11.4 Assumptions and Limitations

- Decision trees produce axis-aligned splits, which may be suboptimal for rotated or highly correlated data
- No explicit distance function is required
- Random forests reduce overfitting relative to single trees

11.5 Divide and Conquer Perspective

Decision trees partition the feature space recursively:

$$\mathbb{R}^d = \bigcup_{m=1}^{M} R_m,$$

where each region R_m corresponds to a leaf.

For regression, predictions are piecewise constant:

$$\hat{f}(x) = \sum_{m=1}^{M} c_m \mathbb{1}(x \in R_m),$$

where

$$c_m = \frac{1}{|R_m|} \sum_{x_i \in R_m} y_i.$$

Remark 11.5.1. This can be interpreted as minimizing within-region variance:

$$\sum_{m=1}^{M} \sum_{x_i \in R_m} (y_i - c_m)^2.$$

11.6 Random Forest Algorithm

Algorithm 11 Random Forest Training

Require: Training data $\mathcal{D}$, number of trees B
Ensure: Ensemble of trees $\{\hat{f}_b\}_{b=1}^{B}$

1: **for** $b = 1$ to B **do**
2: Draw bootstrap sample $\mathcal{D}_b$ from $\mathcal{D}$
3: Train decision tree $\hat{f}_b$:
- At each split, select a random subset of features
- Choose the best split among those features

4: **end for**
5: **return** $\{\hat{f}_b\}_{b=1}^{B}$

Algorithm 12 Random Forest Prediction

Require: Trees $\{\hat{f}_b\}_{b=1}^{B}$, query point x
Ensure: Prediction $\hat{y}$

1: **if** classification **then**
2: $\hat{y} \leftarrow$ majority vote of $\{\hat{f}_b(x)\}$
3: **else**
4: $\hat{y} \leftarrow \frac{1}{B} \sum_{b=1}^{B} \hat{f}_b(x)$
5: **end if**
6: **return** $\hat{y}$

11.7 Implementation Notes

```python
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("rf", RandomForestClassifier())
])

param_grid = {
    "rf__n_estimators": [100, 200],
    "rf__max_depth": [None, 5, 10],
    "rf__max_features": ["sqrt", "log2"]
}

grid = GridSearchCV(pipe, param_grid, cv=5)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_
```

Remark 11.7.1. Random forests typically require minimal preprocessing and perform well out-of-the-box, making them a strong baseline model.

Chapter 12

Logistic Regression

12.1 Binary Classification and the Logistic Function

In binary classification, we model the posterior probability

$$\pi(x) = \mathbb{P}(C_1 \mid x).$$

Definition 12.1.1 (Logistic (Sigmoid) Function)**.** The logistic function is

$$\sigma(t) = \frac{1}{1 + e^{-t}}.$$

Remark 12.1.1. The logistic function maps $\mathbb{R} \to (0, 1)$, making it suitable for modeling probabilities.

12.2 Log-Odds and Linear Model

Define the log-odds (logit) transformation:

$$\log \frac{\mathbb{P}(C_1 \mid x)}{\mathbb{P}(C_0 \mid x)} = \log \frac{\pi(x)}{1 - \pi(x)}.$$

Definition 12.2.1 (Logistic Regression Model)**.** Assume the log-odds are linear in the features:

$$\log \frac{\pi(x)}{1 - \pi(x)} = \beta_0 + \beta^T x.$$

Solving for $\pi(x)$,

$$\pi(x) = \frac{1}{1 + e^{-(\beta_0 + \beta^T x)}}.$$

Remark 12.2.1. This is equivalent to applying the sigmoid function to a linear predictor:

$$\pi(x) = \sigma(\beta_0 + \beta^T x).$$

12.3 Likelihood and Estimation

Given training data

$$\{(x_i, c_i)\}_{i=1}^n, \quad c_i \in \{0, 1\},$$

the likelihood is

$$\mathcal{L}(\beta) = \prod_{i=1}^{n} \pi(x_i; \beta)^{c_i} \left[1 - \pi(x_i; \beta)\right]^{1-c_i}.$$

Definition 12.3.1 (Log-Likelihood)**.** The log-likelihood is

$$\ell(\beta) = \sum_{i=1}^{n} \left[c_i \log \pi(x_i; \beta) + (1 - c_i) \log(1 - \pi(x_i; \beta))\right].$$

Proposition 12.3.1 (Maximum Likelihood Estimation)**.** *The logistic regression estimator is*

$$\hat{\beta} = \arg\max_{\beta} \ell(\beta).$$

12.3.1 Gradient and Optimization

Let $\pi_i = \pi(x_i; \beta)$. Then

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^{n} (c_i - \pi_i) x_i.$$

Proof. We use

$$\frac{d}{dt}\sigma(t) = \sigma(t)(1 - \sigma(t)).$$

Differentiating the log-likelihood,

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^{n} \left[\frac{c_i}{\pi_i} \frac{\partial \pi_i}{\partial \beta} - \frac{1 - c_i}{1 - \pi_i} \frac{\partial \pi_i}{\partial \beta}\right].$$

Using

$$\frac{\partial \pi_i}{\partial \beta} = \pi_i(1 - \pi_i) x_i,$$

we obtain

$$\frac{\partial \ell}{\partial \beta} = \sum_{i=1}^{n} (c_i - \pi_i) x_i.$$

□

Remark 12.3.1. The log-likelihood is concave, so standard optimization methods (gradient ascent, Newton's method) converge to the global optimum.

12.4 Prediction

Definition 12.4.1 (Prediction Rule)**.** Given $\hat{\beta}$, predict

$$\hat{C}(x) = \begin{cases} 1, & \pi(x; \hat{\beta}) \geq 0.5, \\ 0, & \text{otherwise.} \end{cases}$$

12.5 Decision Boundary

The decision boundary is given by

$$\beta_0 + \beta^T x = 0,$$

which is linear in x.

Remark 12.5.1. Thus, logistic regression produces linear decision boundaries, similar to LDA.

12.6 Multiclass Logistic Regression

For K classes, define

$$\pi_k(x) = \mathbb{P}(C_k \mid x).$$

Definition 12.6.1 (Softmax Model)**.**

$$\pi_k(x) = \frac{e^{\beta_k^T x}}{\sum_{j=1}^{K} e^{\beta_j^T x}}.$$

The log-likelihood is

$$\ell(\beta) = \sum_{i=1}^{n} \log \pi_{c_i}(x_i; \beta).$$

Remark 12.6.1. This is known as multinomial logistic regression or softmax regression.

12.7 Discriminative vs Generative Models

Definition 12.7.1 (Discriminative Model)**.** A discriminative model directly estimates

$$\mathbb{P}(C_k \mid x).$$

Definition 12.7.2 (Generative Model)**.** A generative model estimates

$$p(x \mid C_k) \quad \text{and} \quad \mathbb{P}(C_k),$$

and uses Bayes' rule to compute posteriors.

Remark 12.7.1. Logistic regression is discriminative, whereas naive Bayes and LDA/QDA are generative.

12.8 Regularization

To prevent overfitting, add a penalty:

12.8.1 L2 Regularization (Ridge)

$$\hat{\beta} = \arg\max_{\beta} \left[\ell(\beta) - \lambda \|\beta\|_2^2 \right].$$

12.8.2 L1 Regularization (Lasso)

$$\hat{\beta} = \arg\max_{\beta} \left[\ell(\beta) - \lambda \|\beta\|_1 \right].$$

12.9 Algorithm

Algorithm 13 Logistic Regression via Gradient Ascent

Require: Training data $\{(x_i, c_i)\}$, learning rate η
Ensure: Parameter estimate $\hat{\beta}$

1: Initialize β
2: **for** $t = 1$ to max iterations **do**
3: Compute $\pi_i = \sigma(\beta^T x_i)$ for all i
4: Compute gradient:

$$g = \sum_i (c_i - \pi_i) x_i$$

5: Update:

$$\beta \leftarrow \beta + \eta g$$

6: **end for**
7: **return** β

12.10 Implementation Notes

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def logistic_regression_fit(X, y, lr=0.01, n_iter=1000):
    n, d = X.shape
    beta = np.zeros(d)

    for _ in range(n_iter):
        z = X @ beta
```

```
        p = sigmoid(z)
        grad = X.T @ (y - p)
        beta += lr * grad

    return beta

def logistic_regression_predict(X, beta):
    probs = sigmoid(X @ beta)
    return (probs >= 0.5).astype(int)
```

Chapter 13

Support Vector Machines

13.1 Maximum Margin Classification

Support Vector Machines (SVMs) construct a separating hyperplane between classes with maximum margin.

Definition 13.1.1 (Hyperplane)**.** A hyperplane in $\mathbb{R}^d$ is defined as

$$w^T x - b = 0,$$

where $w \in \mathbb{R}^d$ is the normal vector and $b \in \mathbb{R}$ is the bias.

Remark 13.1.1. The sign of $w^T x - b$ determines the predicted class.

13.2 Hard Margin SVM

Assume the data are linearly separable with labels $y_i \in \{-1, 1\}$.

We define two parallel hyperplanes:

$$w^T x - b = 1, \quad w^T x - b = -1.$$

13.2.1 Margin Width

Proposition 13.2.1 (Margin Width)**.** *The distance between the two margin hyperplanes is*

$$\frac{2}{\|w\|_2}.$$

Proof. Let x_0 satisfy

$$w^T x_0 - b = -1.$$

Let $x_1 = x_0 + d\frac{w}{\|w\|}$ lie on the other hyperplane:

$$w^T x_1 - b = 1.$$

Substitute:

$$w^T\left(x_0 + d\frac{w}{\|w\|}\right) - b = w^T x_0 - b + d\frac{w^T w}{\|w\|}.$$

Since $w^T x_0 - b = -1$ and $w^T w = \|w\|^2$,

$$-1 + d\frac{\|w\|^2}{\|w\|} = 1.$$

Thus,

$$-1 + d\,\|w\| = 1 \quad \Rightarrow \quad d = \frac{2}{\|w\|}.$$

□

13.2.2 Optimization Problem

Maximizing the margin is equivalent to minimizing $\|w\|_2^2$:

Definition 13.2.1 (Hard Margin SVM)**.**

$$\min_{w,b} \|w\|_2^2 \quad \text{s.t.} \quad y_i(w^T x_i - b) \geq 1 \quad \forall i.$$

Remark 13.2.1. This is a convex quadratic optimization problem with linear constraints.

13.3 Support Vectors

Definition 13.3.1 (Support Vectors)**.** The support vectors are the data points that lie exactly on the margin:

$$y_i(w^T x_i - b) = 1.$$

Remark 13.3.1. Only support vectors determine the optimal hyperplane. Points farther from the boundary do not affect the solution.

13.4 Soft Margin SVM

When classes are not linearly separable, introduce slack variables $\xi_i \geq 0$.

Definition 13.4.1 (Soft Margin SVM)**.**

$$\min_{w,b,\xi} \quad \|w\|_2^2 + C\sum_{i=1}^{n} \xi_i$$

subject to

$$y_i(w^T x_i - b) \geq 1 - \xi_i, \quad \xi_i \geq 0.$$

Remark 13.4.1. The parameter $C > 0$ controls the tradeoff between margin size and classification errors.

13.4.1 Hinge Loss Formulation

The soft-margin objective can be written as

$$\min_{w,b} \quad \|w\|_2^2 + C \sum_{i=1}^{n} \max\{0,\, 1 - y_i(w^T x_i - b)\}.$$

Remark 13.4.2. The hinge loss penalizes points that are inside the margin or misclassified.

13.5 Kernel Methods

13.5.1 Feature Transformation

Instead of working in the original space, map data to a higher-dimensional feature space:

$$x \mapsto \phi(x).$$

The SVM optimization then uses

$$w^T \phi(x).$$

13.5.2 Kernel Trick

Definition 13.5.1 (Kernel Function)**.** A kernel function is defined as

$$k(x, x') = \phi(x)^T \phi(x').$$

Remark 13.5.1. The kernel trick allows computation in high-dimensional feature spaces without explicitly computing $\phi(x)$.

13.5.3 Common Kernels

- Polynomial kernel:
$$k(x, x') = (x^T x' + c)^d$$
- Radial basis function (RBF):
$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

Remark 13.5.2. RBF kernels produce nonlinear decision boundaries by mapping data into infinite-dimensional feature spaces.

13.6 Dual Formulation (Key Insight)

The SVM solution can be expressed in terms of inner products:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i.$$

Thus predictions become

$$f(x) = \sum_{i=1}^{n} \alpha_i y_i k(x_i, x) - b.$$

Remark 13.6.1. Only support vectors have nonzero α_i, making the model sparse.

13.7 Algorithmic Summary

Algorithm 14 Support Vector Machine (Conceptual)

Require: Training data $\{(x_i, y_i)\}$
Ensure: Classifier (w, b)
1: Solve quadratic optimization problem
2: Identify support vectors
3: Construct decision function:

$$f(x) = \sum_{i} \alpha_i y_i k(x_i, x) - b$$

4: **return** classifier

13.8 Properties and Comparison

- Maximizes geometric margin
- Robust to high-dimensional data
- Depends only on support vectors
- Kernel trick enables nonlinear classification

Remark 13.8.1. Unlike nearest neighbors, SVM does not rely on a distance metric in the original space, but rather on inner products (or kernels).

13.9 Implementation Notes

```python
from sklearn.svm import SVC

model = SVC(kernel="rbf", C=1.0, gamma="scale")
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```python
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC())
])

pipe.fit(X_train, y_train)
```

Remark 13.9.1. SVMs are sensitive to feature scaling, so standardization is typically required.

Chapter 14

k-Means Clustering

14.1 Clustering

Clustering is the task of grouping data points into subsets (clusters) such that points in the same cluster are more similar to each other than to points in other clusters.

- Discover species based on features
- Segment images by pixel similarity
- Group documents by topic

14.2 Types of Clustering Algorithms

14.2.1 Flat (Partitioning) Methods

- Start with an initial partition of the data
- Iteratively refine cluster assignments

14.2.2 Hierarchical Methods

- **Agglomerative (bottom-up)**: merge closest clusters
- **Divisive (top-down)**: split clusters recursively

14.3 k-Means Clustering

14.3.1 Optimization Problem

Given data $\{x_l\}_{l=1}^n$ with $x_l \in \mathbb{R}^d$, partition the data into k clusters $\{C_i\}_{i=1}^k$.

Definition 14.3.1 (k-Means Objective)**.**

$$\hat{C} = \arg\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|_2^2 ,$$

where

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x.$$

Remark 14.3.1. This objective is also called the within-cluster sum of squares (WCSS).

14.3.2 Optimality of the Mean

Proposition 14.3.1. *Given a fixed cluster C, the point minimizing*

$$\sum_{x \in C} \|x - \mu\|_2^2$$

is the mean

$$\mu = \frac{1}{|C|} \sum_{x \in C} x.$$

Proof. Let

$$f(\mu) = \sum_{x \in C} \|x - \mu\|_2^2 .$$

Expanding,

$$f(\mu) = \sum_{x \in C} (x - \mu)^T (x - \mu).$$

Taking gradient,

$$\nabla_\mu f(\mu) = -2 \sum_{x \in C} (x - \mu).$$

Setting equal to zero,

$$\sum_{x \in C} (x - \mu) = 0 \quad \Rightarrow \quad \mu = \frac{1}{|C|} \sum_{x \in C} x.$$

□

14.4 Algorithm

k-means is an iterative algorithm alternating between assignment and update steps.

Algorithm 15 k-Means Clustering

Require: Data $\{x_i\}_{i=1}^n$, number of clusters k
Ensure: Clusters $\{C_i\}$ and centroids $\{\mu_i\}$

Initialize centroids $\mu_1, \ldots, \mu_k$
repeat
  Assignment step:
  for each data point x_i **do**
    Assign x_i to cluster

$$C_j = \arg\min_j \|x_i - \mu_j\|_2$$

  end for
  Update step:
  for each cluster C_j **do**
    Update centroid

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x$$

  end for
until convergence

Remark 14.4.1. Each iteration decreases the objective function, so the algorithm converges to a local minimum.

14.5 Inertia

Definition 14.5.1 (Inertia)**.** Inertia is the total within-cluster sum of squares:

$$\text{Inertia} = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|_2^2 .$$

Remark 14.5.1.

- Lower inertia indicates tighter clusters
- Inertia always decreases as k increases

14.6 Choosing the Number of Clusters

14.6.1 Elbow Method

Plot inertia as a function of k and look for a point where the decrease slows significantly.

Remark 14.6.1. The "elbow" corresponds to a good tradeoff between model complexity and fit.

14.7 Limitations

- Sensitive to initialization
- Converges to local minima
- Assumes spherical clusters (Euclidean distance)
- Sensitive to feature scaling

14.7.1 Initialization Strategies

- Random initialization
- Multiple restarts (parameter `n_init`)
- k-means++ initialization

14.8 Variants

14.8.1 k-Medians

Definition 14.8.1 (k-Medians Objective)**.**

$$\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|_1 ,$$

where μ_i is the coordinate-wise median.

Remark 14.8.1. Using L_1 distance makes the method more robust to outliers.

14.9 Geometric Interpretation

k-means partitions the space into Voronoi regions:

$$R_j = \{x : \|x - \mu_j\|_2 \leq \|x - \mu_l\|_2 \ \forall l\}.$$

Remark 14.9.1. The decision boundaries between clusters are linear (perpendicular bisectors).

14.10 Implementation Notes

```
from sklearn.cluster import KMeans

model = KMeans(n_clusters=3, n_init=10)
model.fit(X)
```

```

labels = model.labels_
centroids = model.cluster_centers_
```

```
import matplotlib.pyplot as plt

inertia = []
k_values = range(1, 10)

for k in k_values:
    model = KMeans(n_clusters=k, n_init=10)
    model.fit(X)
    inertia.append(model.inertia_)

plt.plot(k_values, inertia)
plt.xlabel("k")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```

Chapter 15

Gaussian Mixture Models

15.1 Probabilistic Clustering

Gaussian Mixture Models (GMMs) provide a probabilistic approach to clustering by modeling the data as a mixture of Gaussian distributions.

Definition 15.1.1 (Gaussian Mixture Model)**.** A GMM with K components assumes

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x; \mu_k, \Sigma_k),$$

where

- $\pi_k \geq 0$, $\sum_{k=1}^{K} \pi_k = 1$ (mixture weights)
- $\mu_k \in \mathbb{R}^d$ (means)
- $\Sigma_k \in \mathbb{R}^{d \times d}$ (covariances)

Remark 15.1.1. Each component represents a cluster, but membership is probabilistic rather than deterministic.

15.2 Latent Variables

Introduce latent variables z_{ki}:

$$z_{ki} = \begin{cases} 1, & \text{if } x_i \text{ belongs to component } k, \\ 0, & \text{otherwise.} \end{cases}$$

Since cluster memberships are unknown, we treat z_{ki} as hidden variables.

15.3 Likelihood Function

Given data $\{x_i\}_{i=1}^n$, the likelihood is

$$L(\theta; x) = \prod_{i=1}^{n} \left[\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k) \right].$$

Remark 15.3.1. The log-likelihood involves a log of sums, making direct maximization difficult.

15.4 Expectation-Maximization (EM) Algorithm

The EM algorithm iteratively maximizes the likelihood in the presence of latent variables.

Definition 15.4.1 (EM Framework)**.** Given current parameters θ, compute updated parameters θ' such that

$$p(D \mid \theta') \geq p(D \mid \theta).$$

15.4.1 E-Step

Compute posterior probabilities (responsibilities):

$$\gamma_{ik} = \mathbb{P}(z_{ki} = 1 \mid x_i) = \frac{\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}.$$

Remark 15.4.1. γ_{ik} represents the degree to which point x_i belongs to cluster k.

15.4.2 M-Step

Update parameters using responsibilities:

$$N_k = \sum_{i=1}^{n} \gamma_{ik}.$$

$$\pi_k = \frac{N_k}{n}.$$

$$\mu_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} x_i.$$

$$\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^T.$$

Remark 15.4.2. The M-step computes weighted means and covariances, where weights are the responsibilities.

15.4.3 Algorithm

Algorithm 16 Gaussian Mixture Model via EM

Require: Data $\{x_i\}$, number of components K
Ensure: Parameters $\{\pi_k, \mu_k, \Sigma_k\}$
1: Initialize parameters
2: **repeat**
3: **E-step:** Compute γ_{ik} for all i, k
4: **M-step:** Update π_k, μ_k, Σ_k
5: **until** convergence

15.5 Connection to k-Means

Remark 15.5.1. k-means can be viewed as a limiting case of GMMs:

- Equal covariances $\Sigma_k = \sigma^2 I$
- $\sigma^2 \to 0$

This leads to hard assignments instead of probabilistic memberships.

15.6 Soft Clustering

Definition 15.6.1 (Soft Clustering)**.** In GMMs, each data point belongs to each cluster with probability γ_{ik}.

Remark 15.6.1. This contrasts with k-means, where each point is assigned to exactly one cluster.

15.7 Properties and Limitations

- Captures elliptical clusters via covariance matrices
- Provides probabilistic cluster assignments
- More flexible than k-means
- Sensitive to initialization
- May converge to local optima

15.8 Model Selection

Choosing K:

- Likelihood-based methods
- Information criteria (AIC, BIC)

Remark 15.8.1. Unlike inertia in k-means, likelihood does not necessarily monotonically improve with K when penalization is used.

15.9 Comparison with k-Means

- GMM:
  - Probabilistic model
  - Elliptical clusters
  - Soft assignments
- k-means:
  - Deterministic assignments
  - Spherical clusters
  - Faster and simpler

15.10 Implementation Notes

```
from sklearn.mixture import GaussianMixture

model = GaussianMixture(n_components=3)
model.fit(X)

labels = model.predict(X)
probs = model.predict_proba(X)
```

Remark 15.10.1. `predict_proba` returns the soft cluster assignments γ_{ik}.

15.11 Further Clustering Methods

- Agglomerative clustering (hierarchical)
- DBSCAN (density-based clustering)

Remark 15.11.1. These methods do not require specifying the number of clusters in advance and can capture more complex cluster shapes.

Chapter 16

Spectral Embedding and Spectral Clustering

16.1 Spectral Methods

Spectral methods construct low-dimensional representations of data using eigenvectors of matrices derived from pairwise similarities.

Definition 16.1.1 (Spectral Embedding)**.** Spectral embedding maps data points into a lower-dimensional space using eigenvectors of a graph-based matrix constructed from similarities.

Definition 16.1.2 (Spectral Clustering)**.** Spectral clustering applies a standard clustering method (e.g., k-means) to the embedded coordinates obtained from spectral embedding.

16.2 Graphs

Definition 16.2.1 (Graph)**.** A graph is defined as

$$G = (V, E),$$

where V is a set of vertices and E is a set of edges.

Definition 16.2.2 (Vertex and Edge)**.**

- A **vertex** is a node in the graph
- An **edge** connects two vertices

16.3 Similarity Graphs

Definition 16.3.1 (Similarity Graph)**.** A similarity graph connects data points based on a notion of similarity.

Common constructions:

- ε-neighborhood graph
- k-nearest neighbor graph
- Fully connected graph with weighted edges

Remark 16.3.1. Edges may be weighted by similarity values such as Gaussian kernels:

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right).$$

16.4 Adjacency Matrix

Definition 16.4.1 (Adjacency Matrix)**.** The adjacency matrix $A \in \mathbb{R}^{n\times n}$ is defined by

$$a_{ij} = \begin{cases} 1, & \text{if vertices } i \text{ and } j \text{ are connected,} \\ 0, & \text{otherwise.} \end{cases}$$

Remark 16.4.1. For undirected graphs, A is symmetric:

$$A = A^T.$$

Example 16.4.1 (Adjacency Matrix)**.**

$$A = \begin{pmatrix} 0 & 1 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{pmatrix}.$$

16.5 Degree Matrix and Graph Laplacian

Definition 16.5.1 (Degree Matrix)**.** The degree matrix D is diagonal with

$$D_{ii} = \sum_{j=1}^{n} a_{ij}.$$

Definition 16.5.2 (Graph Laplacian)**.** The (unnormalized) graph Laplacian is

$$L = D - A.$$

Remark 16.5.1. The Laplacian encodes connectivity structure and plays a central role in spectral methods.

16.6 Spectral Embedding

Spectral embedding is based on eigenvectors of the Laplacian.

Definition 16.6.1 (Spectral Embedding Coordinates)**.** Let L have eigenvalues

$$0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$$

with corresponding eigenvectors $v_1, \ldots, v_n$.

The spectral embedding of dimension k is given by

$$Z = \begin{pmatrix} v_2 & v_3 & \cdots & v_{k+1} \end{pmatrix}.$$

Remark 16.6.1. The first eigenvector v_1 is constant and typically discarded.

16.7 Spectral Clustering Algorithm

Algorithm 17 Spectral Clustering

Require: Data $\{x_i\}$, number of clusters k
Ensure: Cluster assignments
1: Construct similarity graph and adjacency matrix A
2: Compute degree matrix D
3: Compute Laplacian $L = D - A$
4: Compute first k eigenvectors of L
5: Form matrix Z from these eigenvectors
6: Apply k-means to rows of Z
7: **return** cluster labels

16.8 Interpretation

- The graph encodes local similarity structure
- Eigenvectors capture global structure
- Clustering is performed in the embedded space

Remark 16.8.1. Spectral methods can identify nonlinearly separable clusters that are not well captured by k-means in the original space.

16.9 Properties of the Laplacian

Proposition 16.9.1. *The Laplacian matrix L is symmetric and positive semidefinite.*

Proof. For any $x \in \mathbb{R}^n$,

$$x^T L x = x^T (D - A)x = \sum_i d_i x_i^2 - \sum_{i,j} a_{ij} x_i x_j.$$

Rewriting,

$$x^T L x = \frac{1}{2} \sum_{i,j} a_{ij} (x_i - x_j)^2 \geq 0.$$

□

Proposition 16.9.2. *The multiplicity of the eigenvalue* 0 *equals the number of connected components in the graph.*

16.10 Implementation Notes

```
from sklearn.cluster import SpectralClustering

model = SpectralClustering(
    n_clusters=3,
    affinity="nearest_neighbors"
)

labels = model.fit_predict(X)
```

Remark 16.10.1. Choice of similarity graph and its parameters (e.g., k in k-NN or σ in RBF kernels) strongly affects performance.

Chapter 17

Laplacian Eigenmaps

17.1 Motivation

Laplacian Eigenmaps is a nonlinear dimensionality reduction technique that preserves local neighborhood structure using a graph representation of the data.

Remark 17.1.1. Unlike PCA, which captures global variance, Laplacian Eigenmaps focuses on preserving local geometry.

17.2 Graph Construction

Given data $\{x_i\}_{i=1}^n \subset \mathbb{R}^d$, construct a similarity graph:

- Connect points using k-nearest neighbors or ε-neighborhoods
- Assign weights w_{ij} to edges

A common choice is the heat kernel:

$$w_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right).$$

Definition 17.2.1 (Weight Matrix)**.** Let $W \in \mathbb{R}^{n\times n}$ be defined by

$$W_{ij} = w_{ij}.$$

17.3 Graph Laplacian

Definition 17.3.1 (Degree Matrix)**.**

$$D_{ii} = \sum_{j=1}^{n} W_{ij}.$$

Definition 17.3.2 (Laplacian Matrix)**.**

$$L = D - W.$$

17.4 Optimization Problem

We seek a low-dimensional embedding $y_i \in \mathbb{R}^m$ for each data point x_i.

Definition 17.4.1 (Laplacian Eigenmaps Objective)**.**

$$\min_{\{y_i\}} \quad \frac{1}{2} \sum_{i,j} W_{ij} \left\| y_i - y_j \right\|_2^2 .$$

Remark 17.4.1. Nearby points (large W_{ij}) are encouraged to remain close in the embedding.

17.4.1 Matrix Form

Let $Y = (y_1, \ldots, y_n)^T \in \mathbb{R}^{n \times m}$. Then

$$\sum_{i,j} W_{ij} \left\| y_i - y_j \right\|_2^2 = 2 \ \mathrm{tr}(Y^T L Y).$$

Proof. Expand:

$$\left\| y_i - y_j \right\|_2^2 = y_i^T y_i - 2 y_i^T y_j + y_j^T y_j.$$

Summing,

$$\sum_{i,j} W_{ij} \left\| y_i - y_j \right\|_2^2 = 2 \sum_i d_i y_i^T y_i - 2 \sum_{i,j} W_{ij} y_i^T y_j.$$

This equals

$$2 \ \mathrm{tr}(Y^T D Y) - 2 \ \mathrm{tr}(Y^T W Y) = 2 \ \mathrm{tr}(Y^T L Y).$$

□

17.5 Constraints

To avoid trivial solutions (e.g., $y_i = 0$), impose constraints:

$$Y^T D Y = I, \quad Y^T D \mathbf{1} = 0.$$

Remark 17.5.1. The second constraint removes the constant eigenvector.

17.6 Solution via Eigenvalue Problem

The optimization reduces to solving the generalized eigenvalue problem:

$$Lv = \lambda Dv.$$

Definition 17.6.1 (Embedding Coordinates)**.** The embedding is given by the eigenvectors corresponding to the smallest nonzero eigenvalues:

$$Y = [v_2, \ldots, v_{m+1}].$$

Remark 17.6.1. The first eigenvector v_1 corresponds to eigenvalue 0 and is constant.

17.7 Interpretation

- Minimizes distortion of local neighborhoods
- Preserves manifold structure
- Produces a low-dimensional embedding reflecting intrinsic geometry

17.8 Connection to Spectral Clustering

Remark 17.8.1. Laplacian Eigenmaps provides the embedding step used in spectral clustering. After computing Y, clustering methods (e.g., k-means) can be applied to the embedded coordinates.

17.9 Algorithm

Algorithm 18 Laplacian Eigenmaps

Require: Data $\{x_i\}$, embedding dimension m
Ensure: Embedded coordinates $\{y_i\}$
1: Construct similarity graph and weight matrix W
2: Compute degree matrix D
3: Compute Laplacian $L = D - W$
4: Solve $Lv = \lambda Dv$
5: Take eigenvectors $v_2, \ldots, v_{m+1}$
6: Form embedding $Y = [v_2, \ldots, v_{m+1}]$
7: **return** $\{y_i\}$

17.10 Properties and Limitations

- Captures nonlinear structure

- Sensitive to graph construction (choice of k, σ)
- Requires eigen-decomposition (computational cost)

17.11 Implementation Notes

```
from sklearn.manifold import SpectralEmbedding

model = SpectralEmbedding(
    n_components=2,
    n_neighbors=10
)

Y = model.fit_transform(X)
```

Remark 17.11.1. The embedding can be visualized or used as input to clustering or classification algorithms.

Chapter 18

Manifold Learning

18.1 Motivation

Manifold learning assumes that high-dimensional data lie on or near a low-dimensional manifold embedded in $\mathbb{R}^D$.

Remark 18.1.1. The goal is to recover intrinsic low-dimensional coordinates while preserving geometric structure (distances, neighborhoods, or local linearity).

18.2 Metric Multidimensional Scaling (MDS)

18.2.1 Problem Formulation

Given a distance matrix $\{d_{ij}\}$, find points $\{x_i\}_{i=1}^n \subset \mathbb{R}^d$ such that

$$d_{ij} \approx \|x_i - x_j\|_2 .$$

Definition 18.2.1 (MDS Stress Function)**.**

$$\min_{\{x_i\}} \quad \sum_{i<j} \left(\|x_i - x_j\|_2 - d_{ij} \right)^2 .$$

18.2.2 Classical MDS

Define the squared distance matrix $D^{(2)}$ with entries d_{ij}^2. Let

$$J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^T.$$

Definition 18.2.2 (Centered Gram Matrix)**.**

$$B = -\frac{1}{2} J D^{(2)} J.$$

Proposition 18.2.1. *If $B = XX^T$, then the rows of X provide the embedding coordinates.*

Remark 18.2.1. The embedding is obtained via eigen-decomposition:

$$B = U\Lambda U^T, \quad X = U_d \Lambda_d^{1/2}.$$

18.3 Isomap

18.3.1 Idea

Isomap extends MDS by replacing Euclidean distances with geodesic distances on a neighborhood graph.

Remark 18.3.1. Geodesic distances approximate distances along the manifold rather than through ambient space.

18.3.2 Algorithm

Algorithm 19 Isomap

Require: Data $\{x_i\}$, neighborhood size (k or ε), embedding dimension d
Ensure: Embedded coordinates

1: Construct neighborhood graph:
   - Connect k nearest neighbors or points within ε

2: Initialize graph distances:

$$d_G(i,j) = \begin{cases} \|x_i - x_j\|_2, & \text{if connected,} \\ \infty, & \text{otherwise.} \end{cases}$$

3: Compute shortest paths:

$$d_G(i,j) = \min_k \{d_G(i,j),\ d_G(i,k) + d_G(k,j)\}$$

4: Apply classical MDS to $\{d_G(i,j)\}$
5: **return** embedding

Remark 18.3.2. Shortest paths can be computed using Floyd–Warshall or Dijkstra's algorithm.

18.4 Locally Linear Embedding (LLE)

18.4.1 Idea

LLE assumes each point can be reconstructed as a linear combination of its neighbors.

Definition 18.4.1 (Reconstruction Weights)**.** For each point x_i, find weights w_{ij} solving

$$\min_{w_{ij}} \quad \left\| x_i - \sum_{j \in \mathcal{N}(i)} w_{ij} x_j \right\|_2^2$$

subject to

$$\sum_{j \in \mathcal{N}(i)} w_{ij} = 1.$$

18.4.2 Embedding Step

Find low-dimensional points $\{y_i\}$ minimizing

$$\sum_i \left\| y_i - \sum_j w_{ij} y_j \right\|_2^2.$$

Remark 18.4.1. The weights w_{ij} are fixed from the original space and reused in the embedding space.

18.4.3 Solution

This reduces to an eigenvalue problem:

$$M = (I - W)^T (I - W),$$

and the embedding is given by the eigenvectors corresponding to the smallest nonzero eigenvalues.

18.5 Comparison of Methods

- MDS:
  - Preserves pairwise distances
  - Sensitive to noise in distances
- Isomap:
  - Preserves geodesic distances
  - Captures global manifold structure
- LLE:
  - Preserves local linear structure
  - Does not explicitly preserve distances

18.6 Other Embedding Methods

18.6.1 Random Projection

Project data using a random matrix:

$$y = Rx,$$

where R has random entries.

Remark 18.6.1. Preserves distances approximately (Johnson–Lindenstrauss lemma).

18.6.2 PCA Projection

Project onto top principal components:

$$y = E_K^T x.$$

18.6.3 Spectral Embedding

Uses eigenvectors of a graph Laplacian to embed data.

18.6.4 t-SNE

Remark 18.6.2. t-SNE maps high-dimensional data to low dimensions by preserving local similarities using probability distributions.

18.7 Implementation Notes

```
from sklearn.manifold import MDS, Isomap, LocallyLinearEmbedding

mds = MDS(n_components=2)
X_mds = mds.fit_transform(X)

iso = Isomap(n_neighbors=10, n_components=2)
X_iso = iso.fit_transform(X)

lle = LocallyLinearEmbedding(n_neighbors=10, n_components=2)
X_lle = lle.fit_transform(X)
```

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2)
X_tsne = tsne.fit_transform(X)
```

Remark 18.7.1. Choice of method depends on whether global or local structure is more important.

Chapter 19

SQLite Databases and SQL

19.1 Overview

Relational databases store data in structured tables and are queried using Structured Query Language (SQL).

Remark 19.1.1. SQLite is a lightweight, self-contained database engine that can be embedded directly in applications (e.g., Python).

19.2 Database Schema

We consider a synthetic measurement database with four tables:

- **Data(X, Y, RunID, ID)**: measurement points
- **Runs(RunID, UserID, InstrumentID)**: run metadata
- **Users(UserID, Name)**: user information
- **Instruments(InsID, Name)**: instrument information

Remark 19.2.1. Tables are related via keys:

- RunID links Data and Runs
- UserID links Runs and Users
- InstrumentID links Runs and Instruments

19.3 Basic SQL Queries

19.3.1 Selecting Data

```sql
SELECT * FROM Data;
```

```sql
SELECT X, Y FROM Data WHERE X > 0;
```

19.3.2 Sorting

```sql
SELECT * FROM Data
ORDER BY X DESC;
```

19.4 Aggregation

Definition 19.4.1 (Aggregate Functions)**.** SQL provides functions that summarize data:

- COUNT
- SUM
- AVG
- MIN
- MAX

Example 19.4.1.

```sql
SELECT COUNT(*) FROM Data;

SELECT AVG(X), AVG(Y) FROM Data;
```

19.4.1 Grouping

```sql
SELECT RunID, COUNT(*) AS n_points
FROM Data
GROUP BY RunID;
```

Remark 19.4.1. GROUP BY partitions rows and applies aggregate functions within each group.

19.5 Joins

Definition 19.5.1 (Join)**.** A join combines rows from multiple tables based on matching keys.

19.5.1 Inner Join

```
SELECT *
FROM Data d
JOIN Runs r ON d.RunID = r.RunID;
```

19.5.2 Multiple Joins

```
SELECT u.Name, i.Name, COUNT(*) AS n
FROM Data d
JOIN Runs r ON d.RunID = r.RunID
JOIN Users u ON r.UserID = u.UserID
JOIN Instruments i ON r.InstrumentID = i.InsID
GROUP BY u.Name, i.Name;
```

Remark 19.5.1. Joins enable combining relational data across tables.

19.6 Subqueries

Definition 19.6.1 (Subquery)**.** A subquery is a query nested inside another query.

Example 19.6.1.

```
SELECT *
FROM Data
WHERE RunID IN (
    SELECT RunID FROM Runs WHERE UserID = 1
);
```

19.7 Common Table Expressions (CTEs)

Definition 19.7.1 (CTE)**.** A Common Table Expression defines a temporary result set for use within a query.

Example 19.7.1.

```
WITH RunCounts AS (
    SELECT RunID, COUNT(*) AS n
    FROM Data
    GROUP BY RunID
)
SELECT *
FROM RunCounts
WHERE n > 10;
```

Remark 19.7.1. CTEs improve readability and modularity of complex queries.

19.8 Relational Algebra Perspective

SQL operations correspond to relational algebra:

- SELECT (projection): choose columns
- WHERE (selection): filter rows
- JOIN: combine relations
- GROUP BY: aggregation

Remark 19.8.1. Relational algebra provides a formal foundation for database queries.

19.9 Table Creation and Modification

19.9.1 Creating Tables

```
CREATE TABLE Users (
    UserID INTEGER PRIMARY KEY,
    Name TEXT
);
```

19.9.2 Inserting Data

```
INSERT INTO Users (UserID, Name)
VALUES (1, 'Alice');
```

19.9.3 Updating Data

```
UPDATE Users
SET Name = 'Bob'
WHERE UserID = 1;
```

19.9.4 Deleting Data

```
DELETE FROM Users
WHERE UserID = 1;
```

19.10 Transactions

Definition 19.10.1 (Transaction)**.** A transaction is a sequence of operations executed as a single unit.

```sql
BEGIN TRANSACTION;

INSERT INTO Data (X, Y, RunID)
VALUES (1.0, 2.0, 3);

COMMIT;
```

Remark 19.10.1. Transactions ensure:

- Atomicity
- Consistency
- Isolation
- Durability (ACID properties)

19.11 Python Integration

```python
import sqlite3

conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

cursor.execute("SELECT * FROM Data")
rows = cursor.fetchall()

conn.close()
```

Remark 19.11.1. SQLite integrates seamlessly with Python for building lightweight data pipelines and experiments.